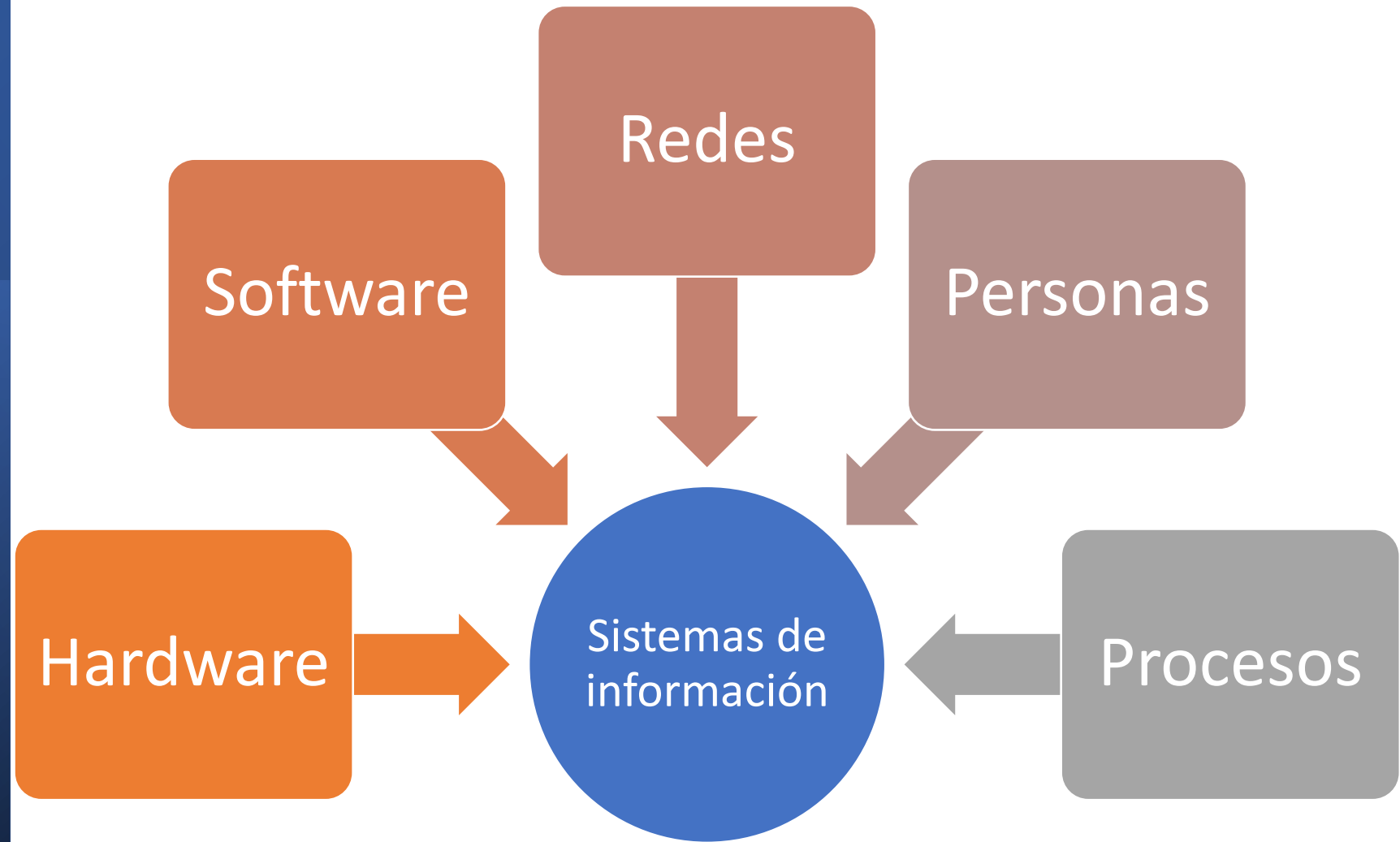




UNIVERSIDAD NACIONAL
RAÚL SCALABRINI ORTIZ
SAN ISIDRO

Componentes

Tecnología
Personas
Procesos



Software:

- Programa o colección de programas , es decir conjunto de instrucciones que le dicen al hardware qué es lo que tiene que hacer. Sin el software, el hardware no sería funcional.

Sistemas de Información

Personas – Procesos

SISTEMA

USUARIO

APLICACIONES

Información
calidad

Datos

SISTEMA OPERATIVO

HARDWARE



DESARROLLO DE SISTEMAS

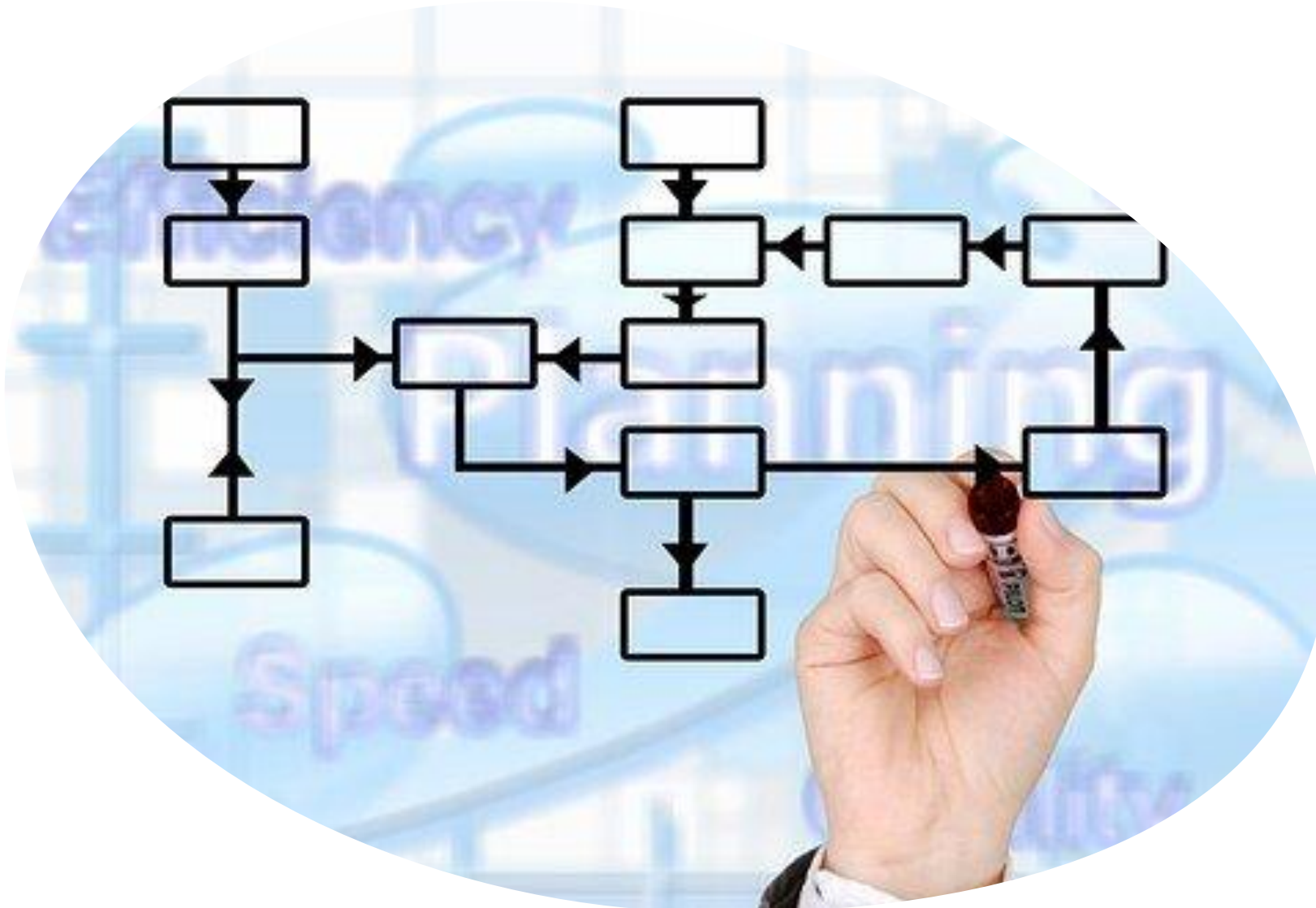
Concretar nuevas ideas y funciones o cubrir nuevas necesidades
Qué necesita el cliente? Cómo se lleva adelante? En qué entorno? Con qué presupuesto? Cómo es el negocio?

FACTORES DE EXITO

- *Objetivos bien definidos*
- *Testeo de factibilidad detallado*
- *Soporte de las autoridades*
- *Compromiso y participación del usuario*
- *Análisis riguroso, para asegurar requerimientos en detalle y sin ambigüedades*
- *Diseño minucioso para asegurar eficiencia, calidad y mantenimiento*
- *Gestión del Proyecto para llevarlo en tiempo y forma*

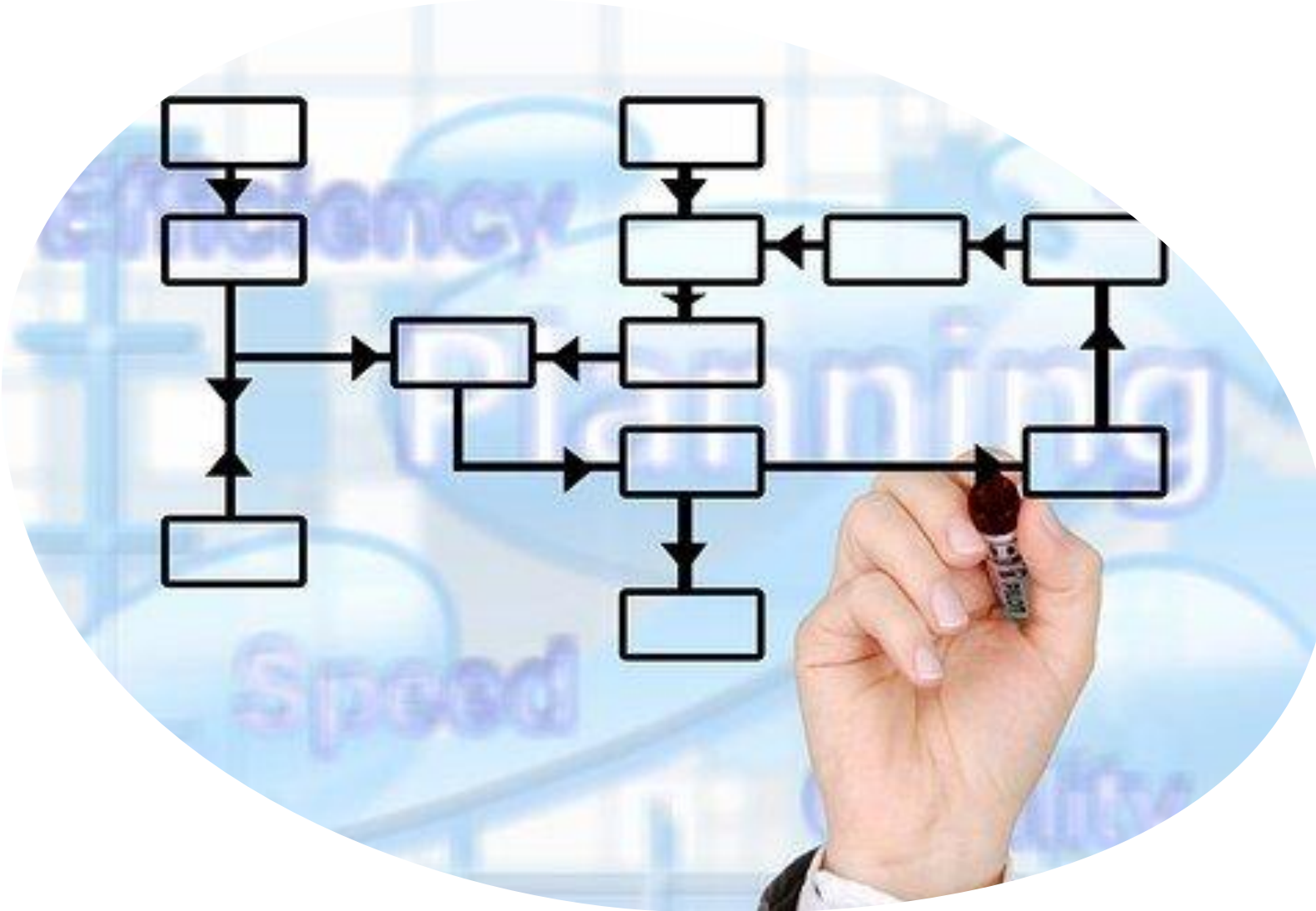
METODOLOGÍA

PROCESOS



Serie de
tareas que se
completan
para lograr
un objetivo o
cumplir con
su misión

PROCESO DE NEGOCIO



Conjunto de actividades de negocio desarrolladas por humanos y/o un Sistema de información para cumplir con un objetivo

Implementación de un Software de Gestión Comercial

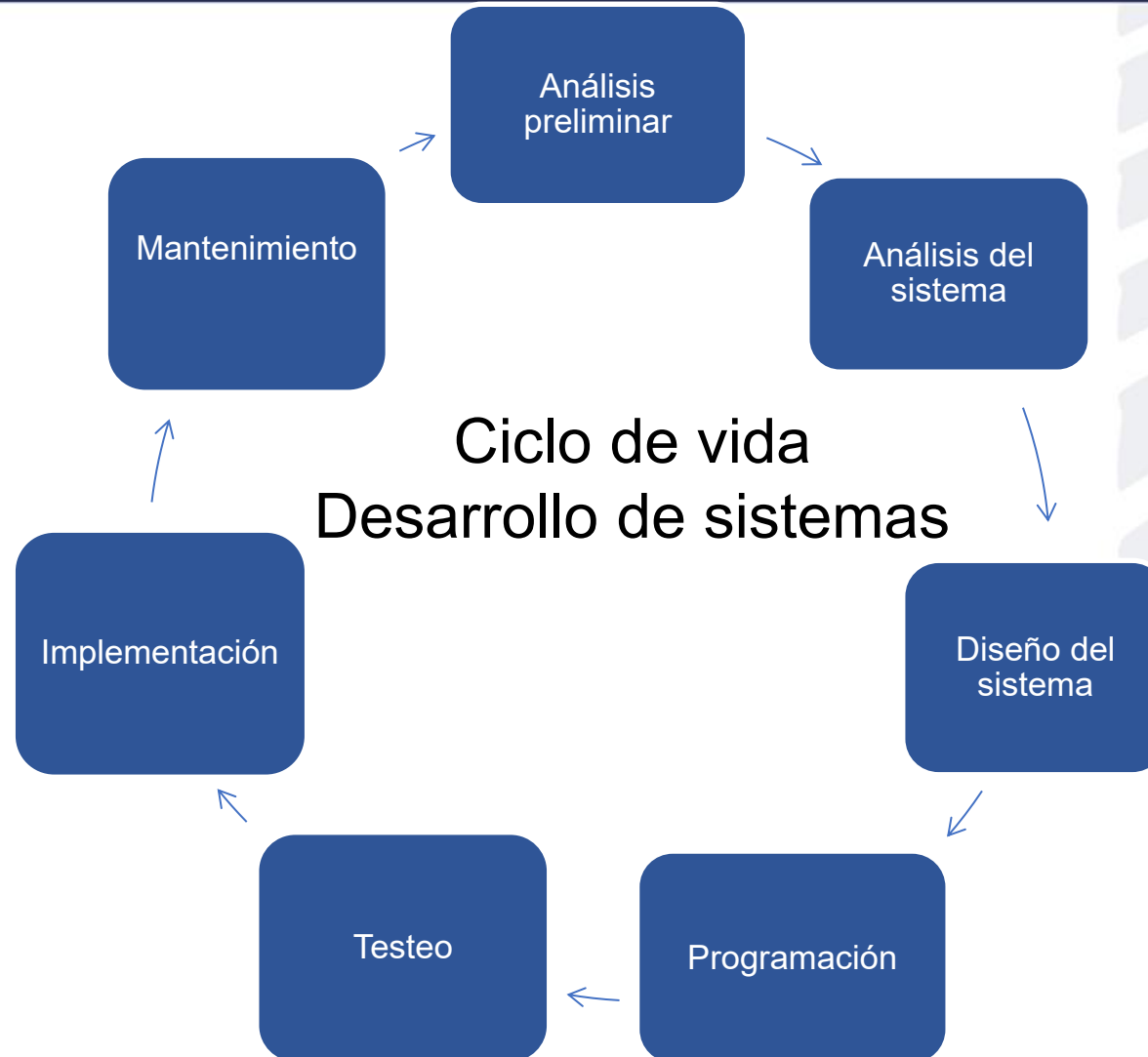
Ejemplo: Un Comerciante que se dedica al rubro pinturería, debido al crecimiento de su negocio en el último año, decide consultar a un especialista para desarrollar un software de gestión para hacer una administración mas eficiente.

En principio, el comerciante sabe que ciertas tareas requieren optimización:

- Llevar actualizado el stock de la mercadería
- Automatizar la facturación, hoy genera las facturas a mano y tiene que calcular los impuestos.
- Agregar más medios de pago y saber cuanto vendió por tipo de medio de pago, efectivo, tarjeta de debito, de crédito , etc.
- Tener actualizado el estado del stock por proveedores, tipo de pinturas y/o colores.
- Y otras más.

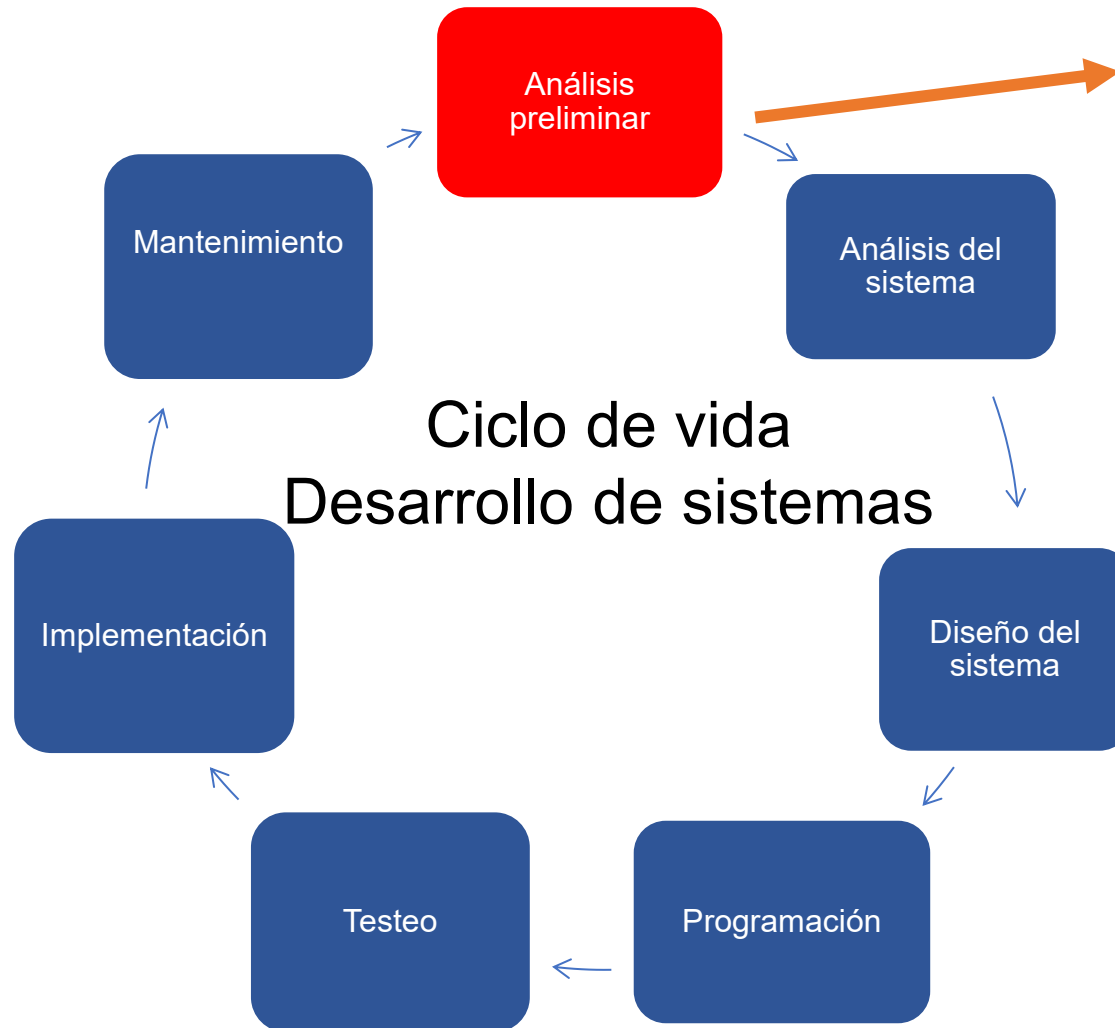
Cómo arrancar con este proyecto?

CICLO DE VIDA ETAPAS DEL DESARROLLO



El Sistema de
Información
debe
considerar los
PROCESOS
y los **DATOS**

CICLO DE VIDA ETAPAS DEL DESARROLLO

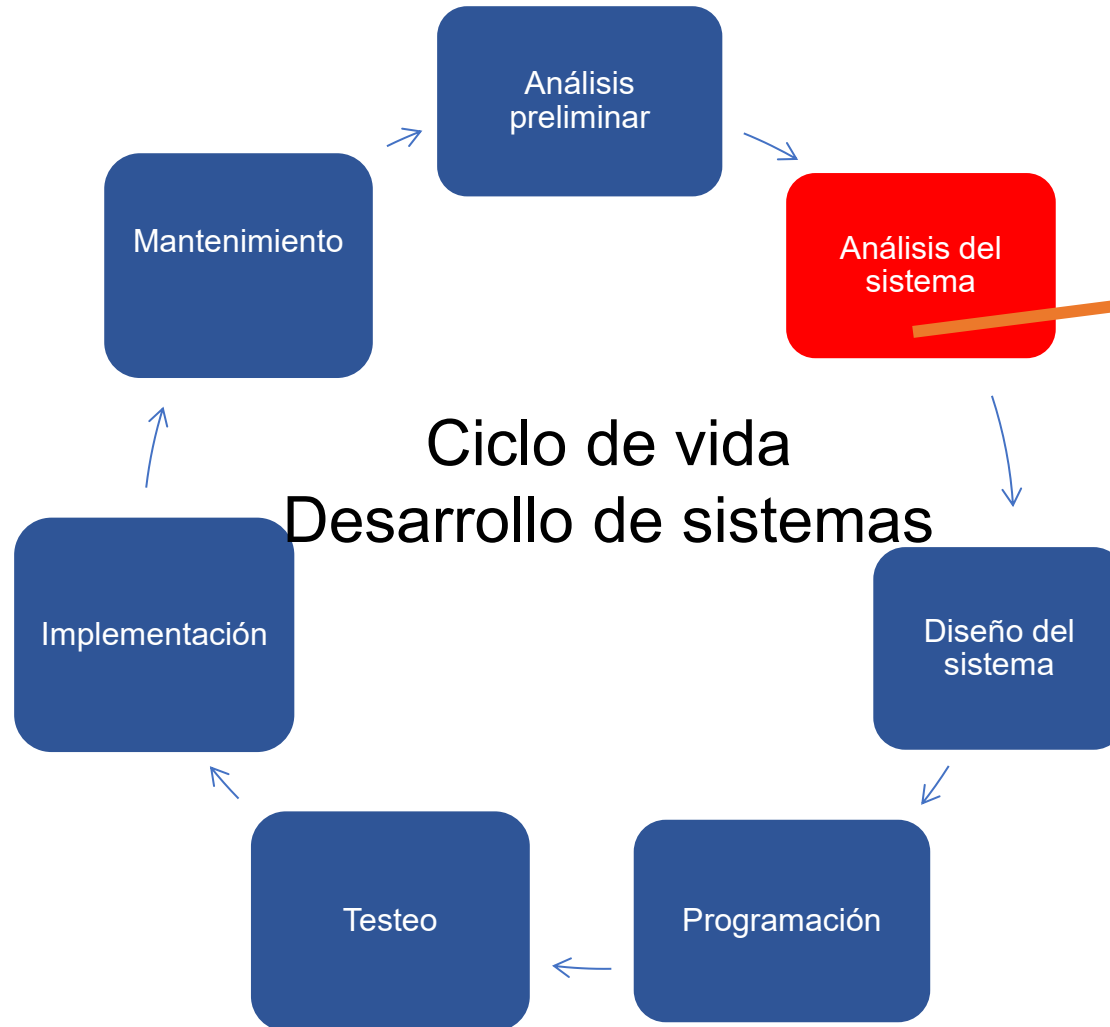


Definición del problema, alcance y datos sobre los objetivos.

- *Qué problema tenemos que resolver*
- *Qué alternativas existen*
- *Qué se hace en otras organizaciones*

*Evaluación de riesgos,
Análisis de factibilidad. Podemos iniciar el proyecto? Se puede hacer realidad?*

CICLO DE VIDA ETAPAS DEL DESARROLLO

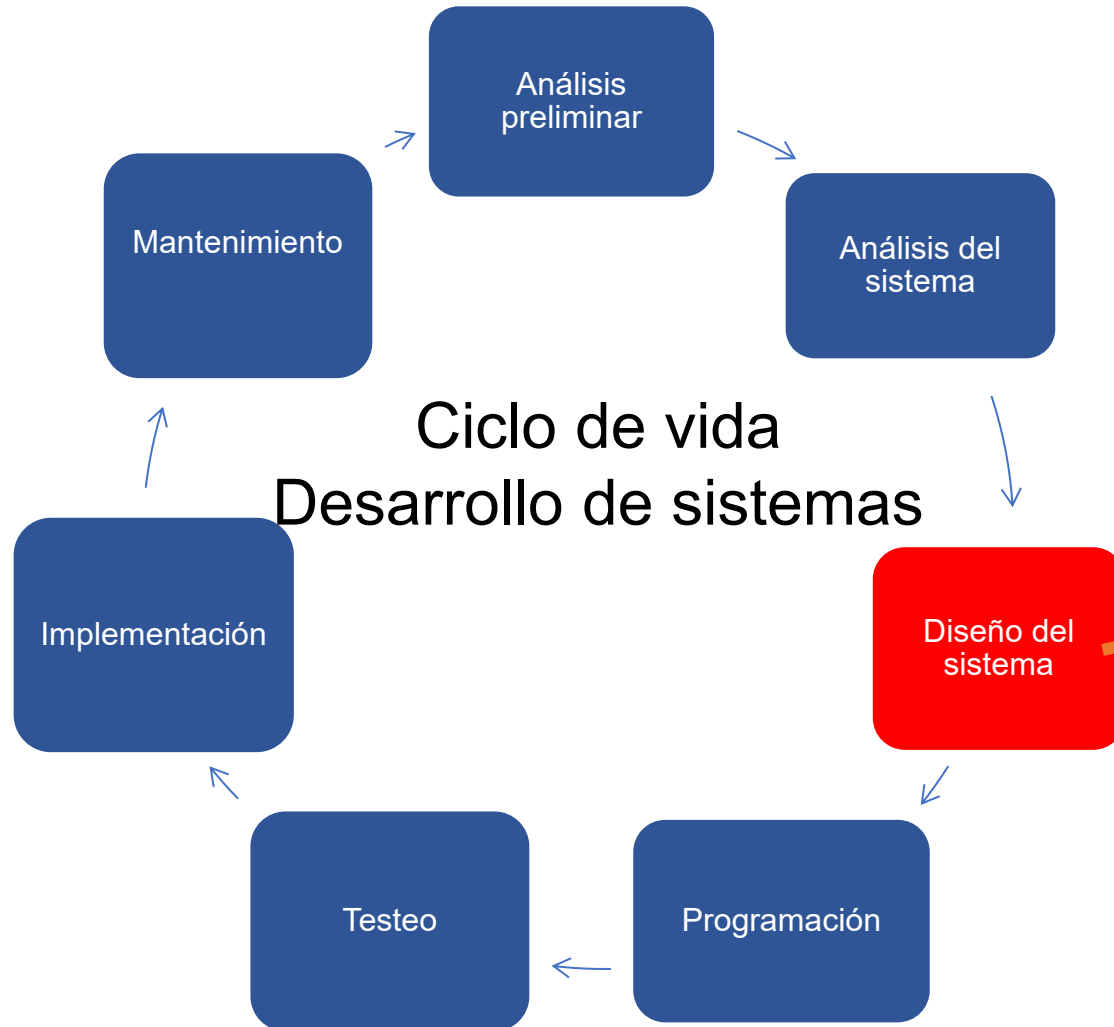


Definición de los requisitos específicos para el nuevo sistema. Qué esperan que haga el sistema

- *Documentar procedimientos*
- *Entrevistas a usuarios/participantes relevantes*
- *Requisitos de datos*

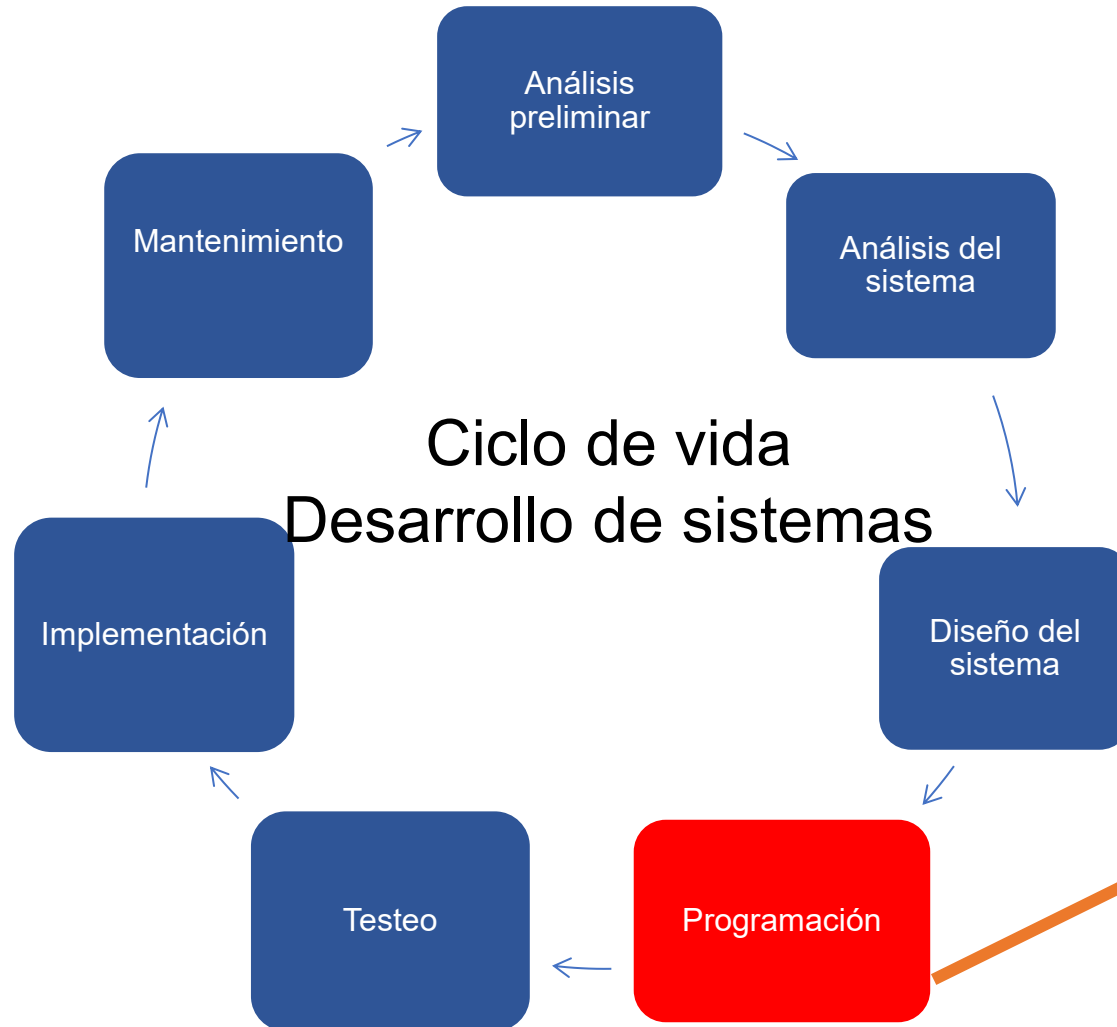
Documento de requisitos

CICLO DE VIDA ETAPAS DEL DESARROLLO



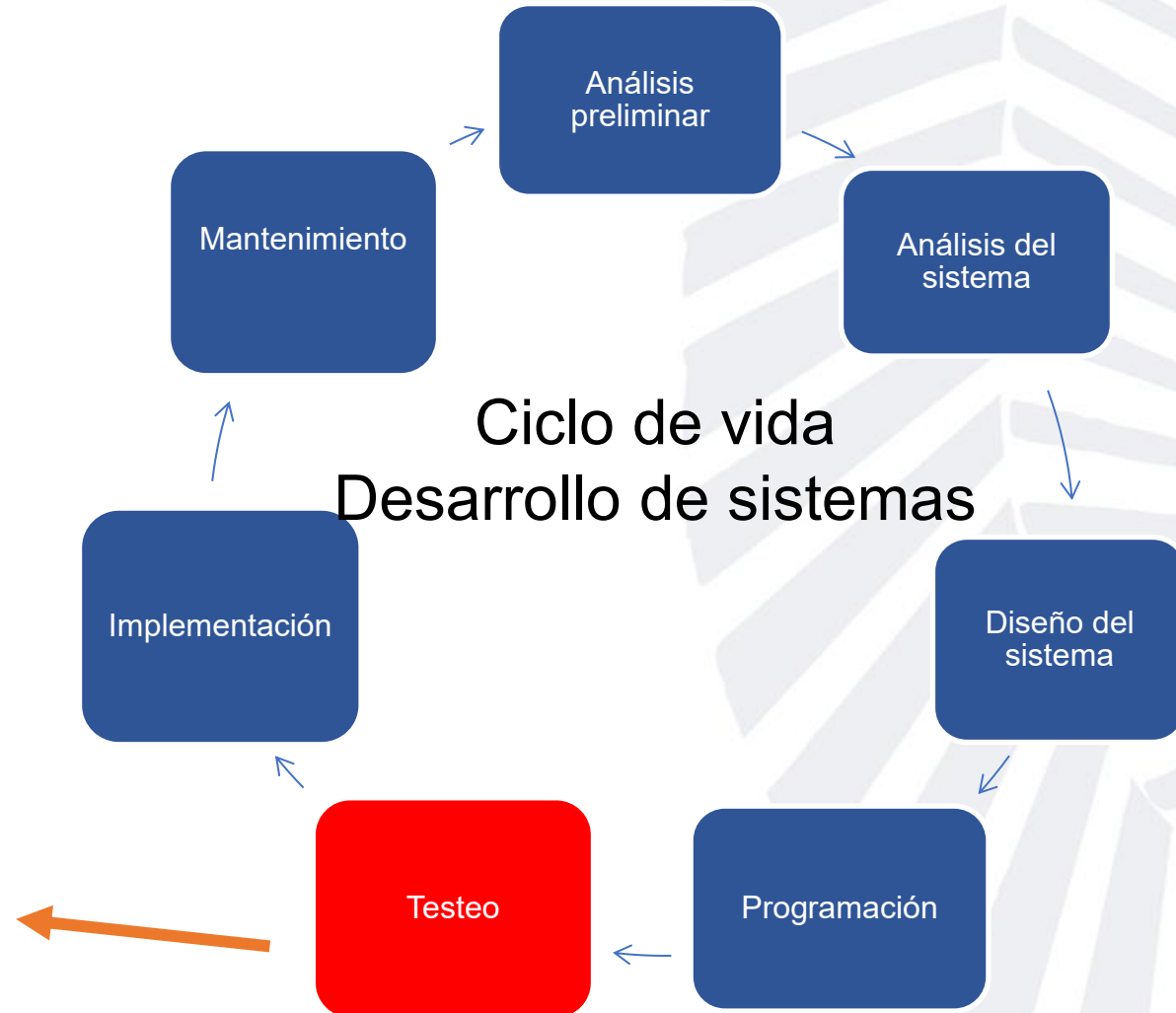
Los requisitos del sistema se traducen a requisitos tecnológicos específicos. Interfase de usuarios, base de datos, servidores, entradas y salidas de datos, reportes, etc. Se obtiene documento de diseño

CICLO DE VIDA ETAPAS DEL DESARROLLO



*Se procede a codificar-desarrollar el software en base al documento de diseño
Se obtiene un programa inicial que cubre los requerimientos del sistema y los de diseño.*

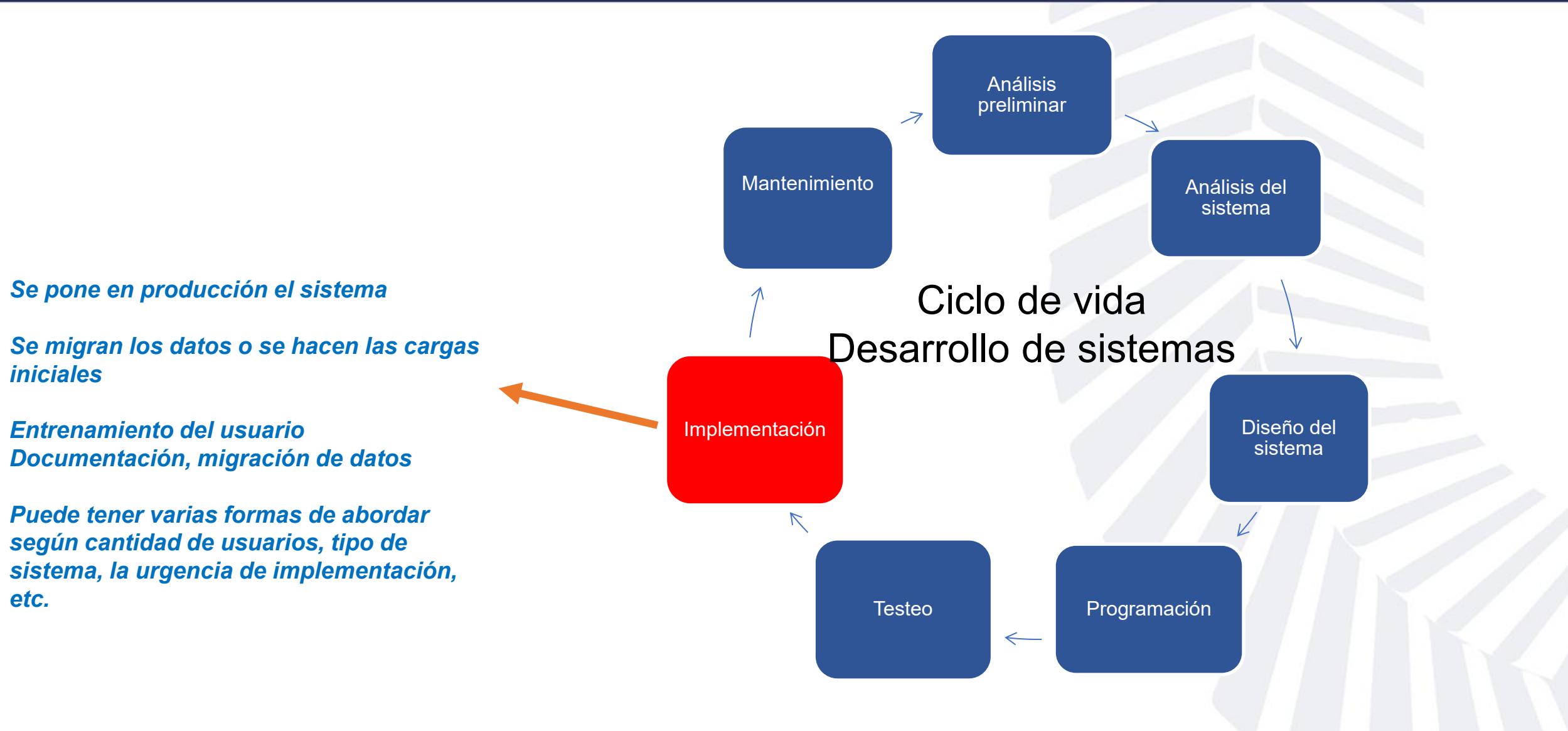
CICLO DE VIDA ETAPAS DEL DESARROLLO



Se procede a testear el sw programado. Se detectan y arreglan errores, bugs y problemas

*Test unitario
Test del sistema
Test de aceptación del usuario*

CICLO DE VIDA ETAPAS DEL DESARROLLO



CICLO DE VIDA ETAPAS DEL DESARROLLO

Se brinda soporte al funcionamiento

Se arreglan los bugs

*Se analizan e implementan nuevos
requerimientos*

Actualizaciones, nuevas versiones.



METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

- **FACTIBILIDAD**

- Investigar los objetivos y alcances
- Económica, técnica y operativa

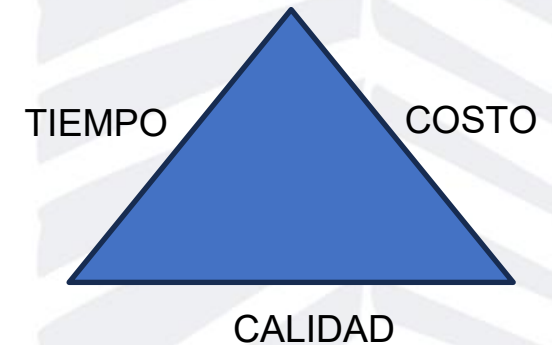
- **DISEÑO**

- Usabilidad, diseño grafico, look and feel, costos, tiempos.

- **COMPRAR O CONSTRUIR**

- **IMPLEMENTACION Y MANTENIMIENTO**

- Transición directa (direct cutover), Implementación Piloto, Operación paralela, Implementación en fases.

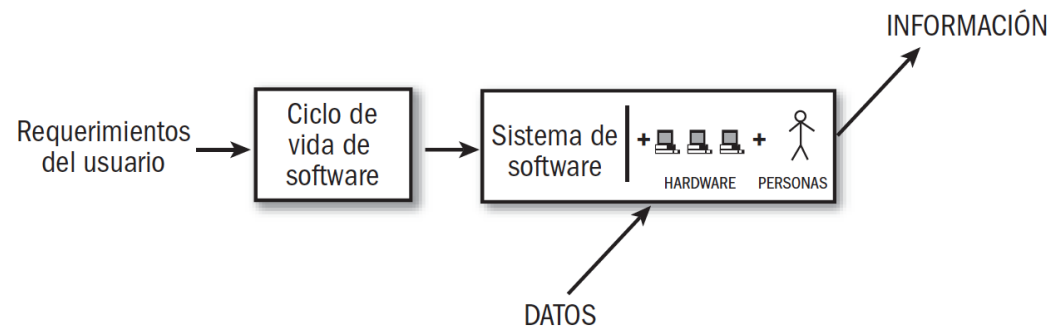


METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE



METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

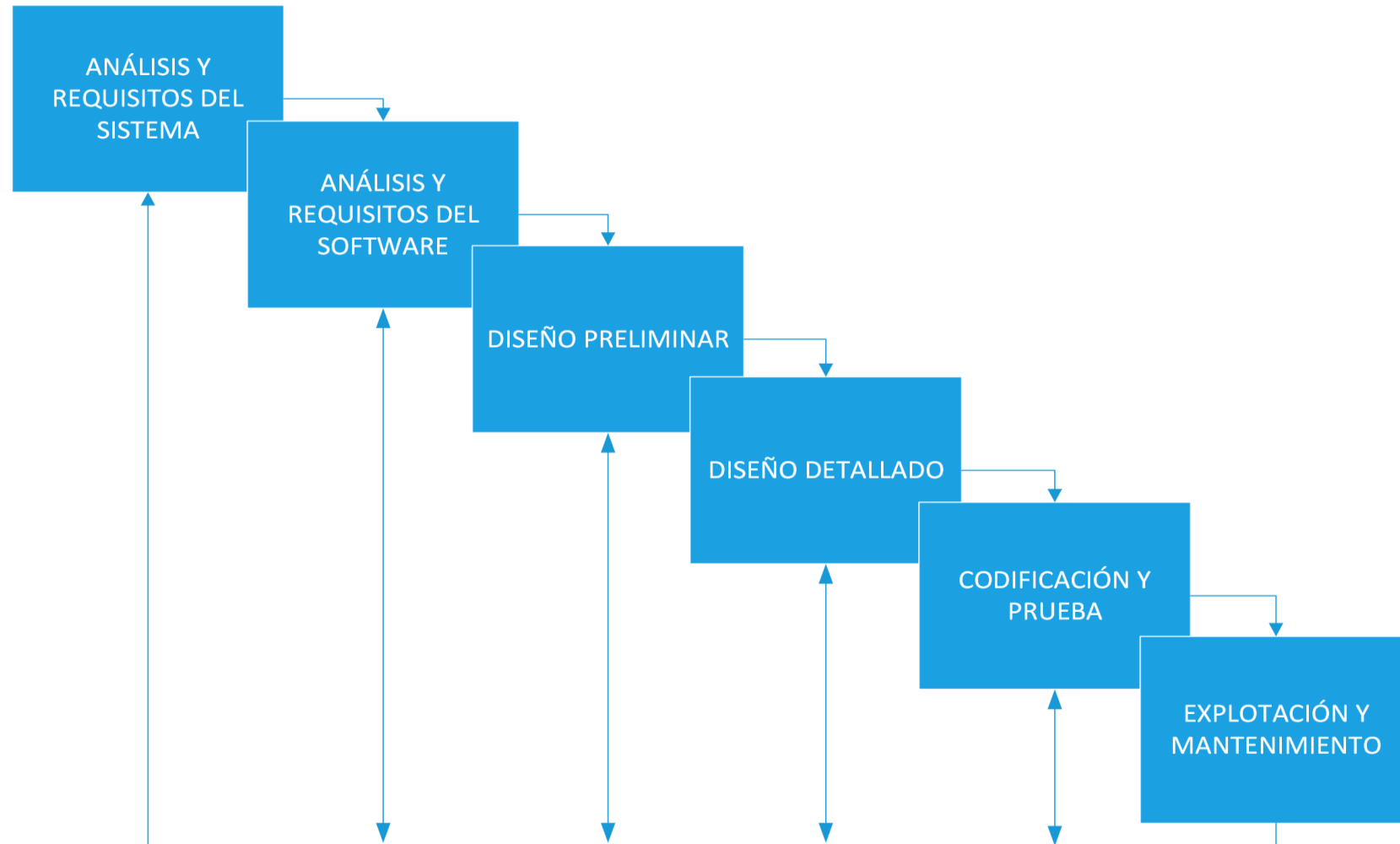
Son los procesos a seguir sistemáticamente para idear, desarrollar, implementar y mantener un producto de software desde que surge la necesidad hasta que se cumple el objetivo por el que fue creado.



III ISO12207

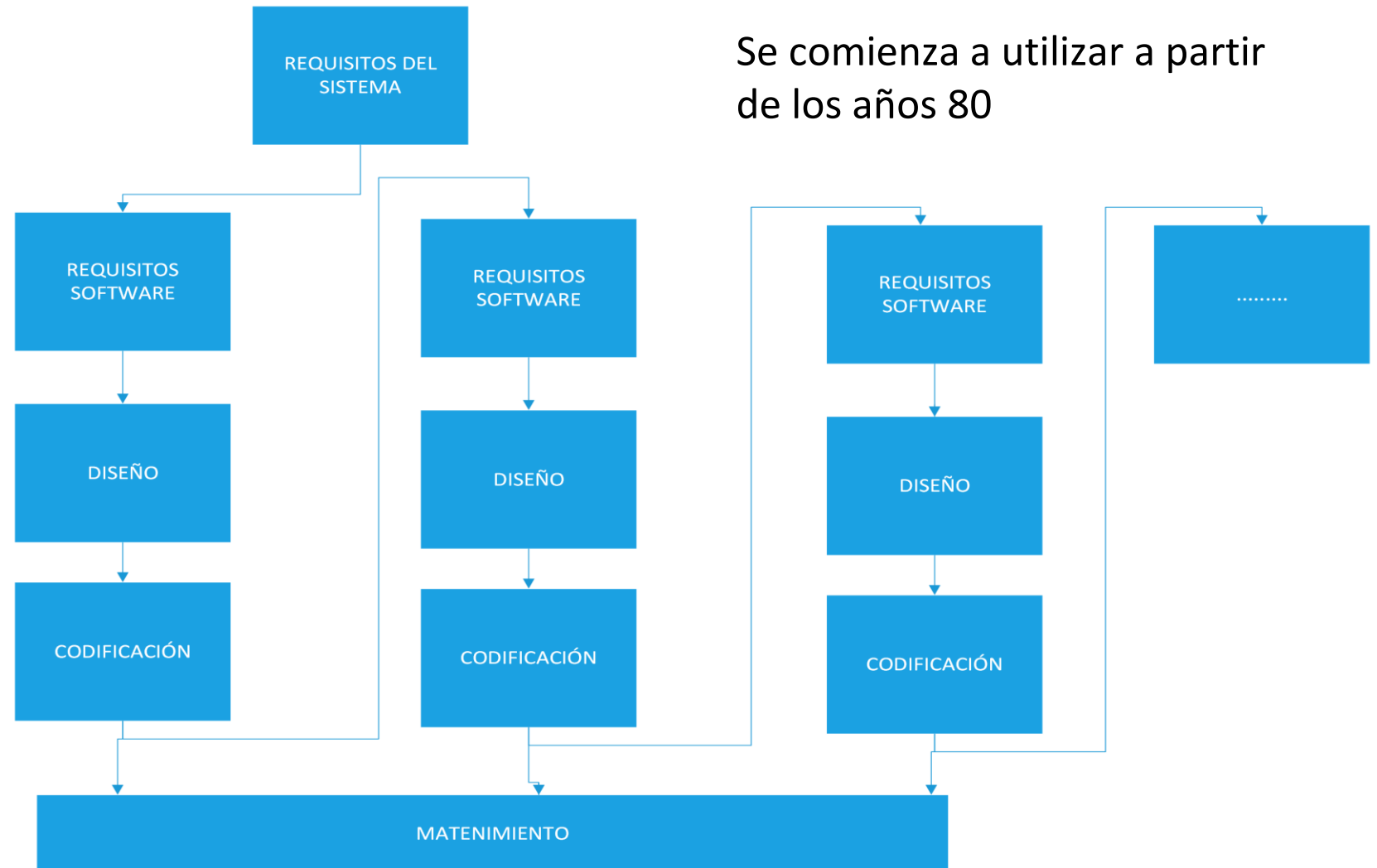
La **ISO**, International Organization for Standardization, en su norma **12207** define al **ciclo de vida de un software** como un marco de referencia que contiene las actividades y las tareas involucradas en el desarrollo, la explotación y el mantenimiento de un producto software, abarcando desde la definición hasta la finalización de su uso.

Metodologías
para el
Desarrollo de
Software



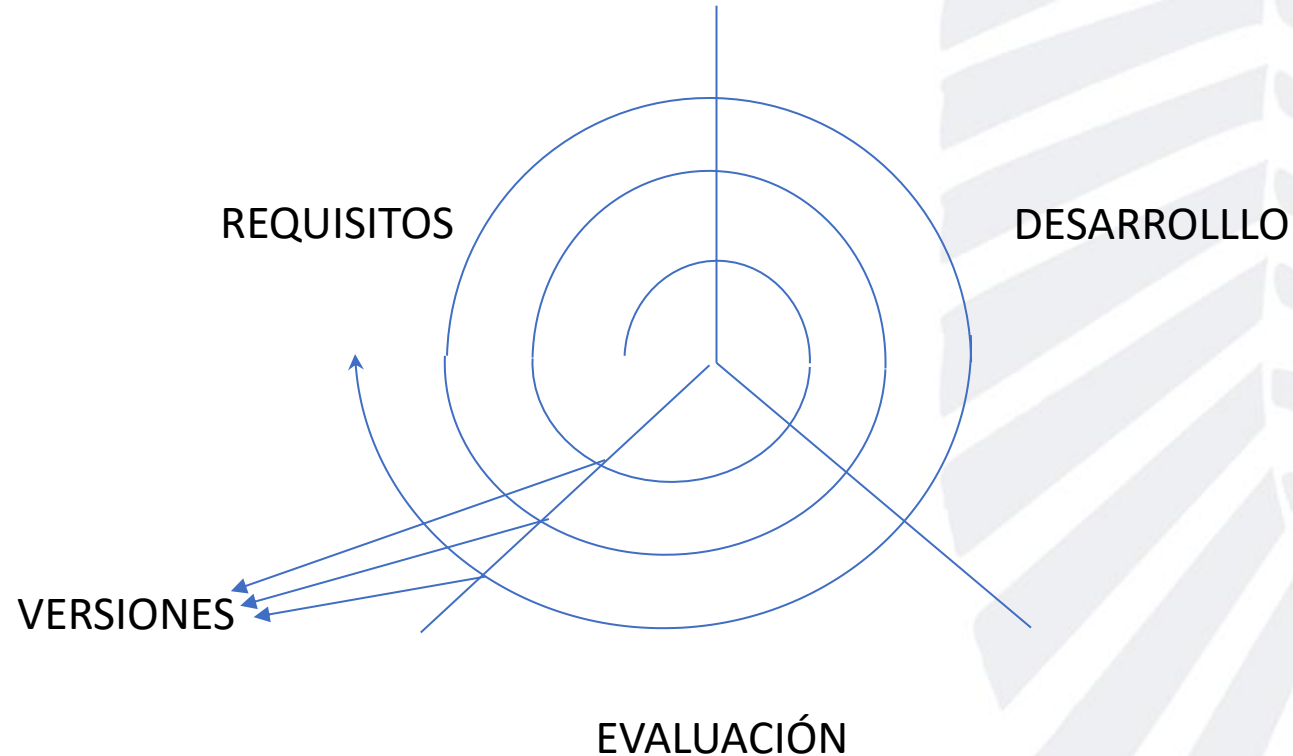
- Características del Modelo CASCADA:
 - Se tarda mucho tiempo en pasar por todo el ciclo
 - Fácil de fracasar por la comunicación con el usuario final
 - El mantenimiento se realiza en el código fuente
 - Las revisiones de proyectos de gran complejidad son muy difíciles
 - Impone una estructura de gestión de proyectos

Metodologías para el Desarrollo de Software



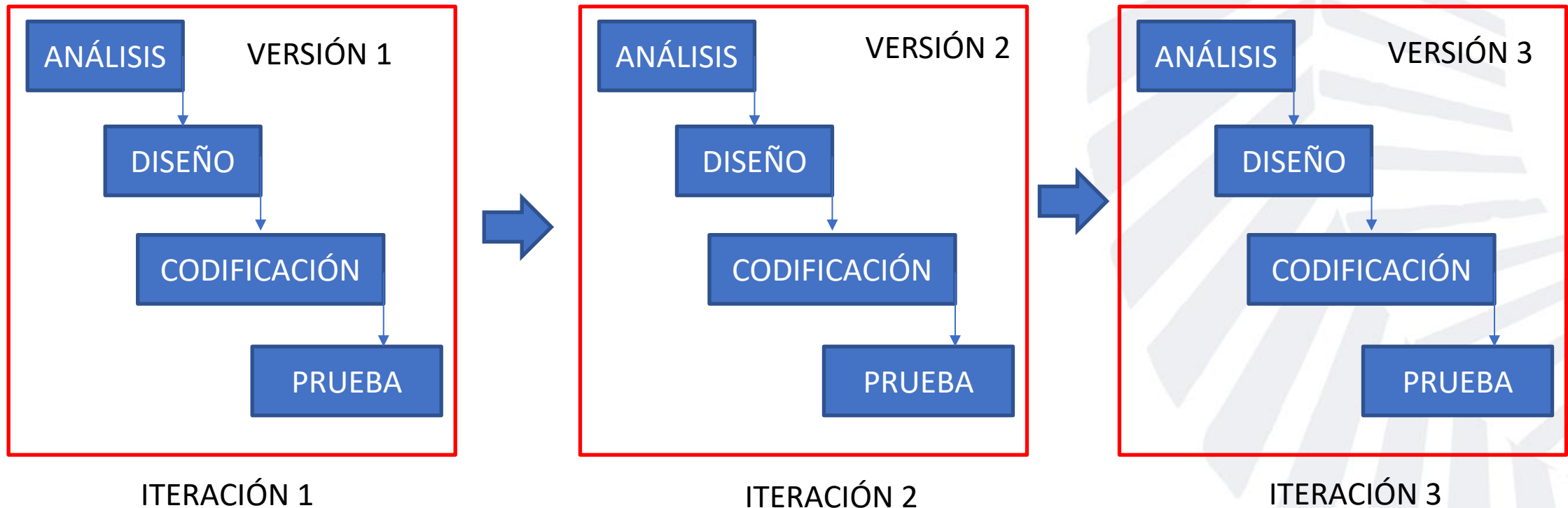
- Características del Modelo INCREMENTAL:
 - Se evitan proyectos largos y se entrega “Algo de valor” a los usuarios con cierta frecuencia
 - El usuario se involucra más.
 - Difícil de evaluar el costo total.
 - Difícil de aplicar a sistemas transaccionales que tienden a ser integrados y a operar como un todo.
 - Requiere gestores experimentados.
 - Los errores en los requisitos se detectan tarde.
 - El resultado puede ser muy positivo.

Metodologías
para el
Desarrollo de
Software



Los requerimientos del usuario pueden cambiar en cualquier momento.
Esto se afronta mediante la iteración de ciclos

CICLO DE VIDA EVOLUTIVO



CICLO DE VIDA EVOLUTIVO

Qué es el método de desarrollo por prototipos?

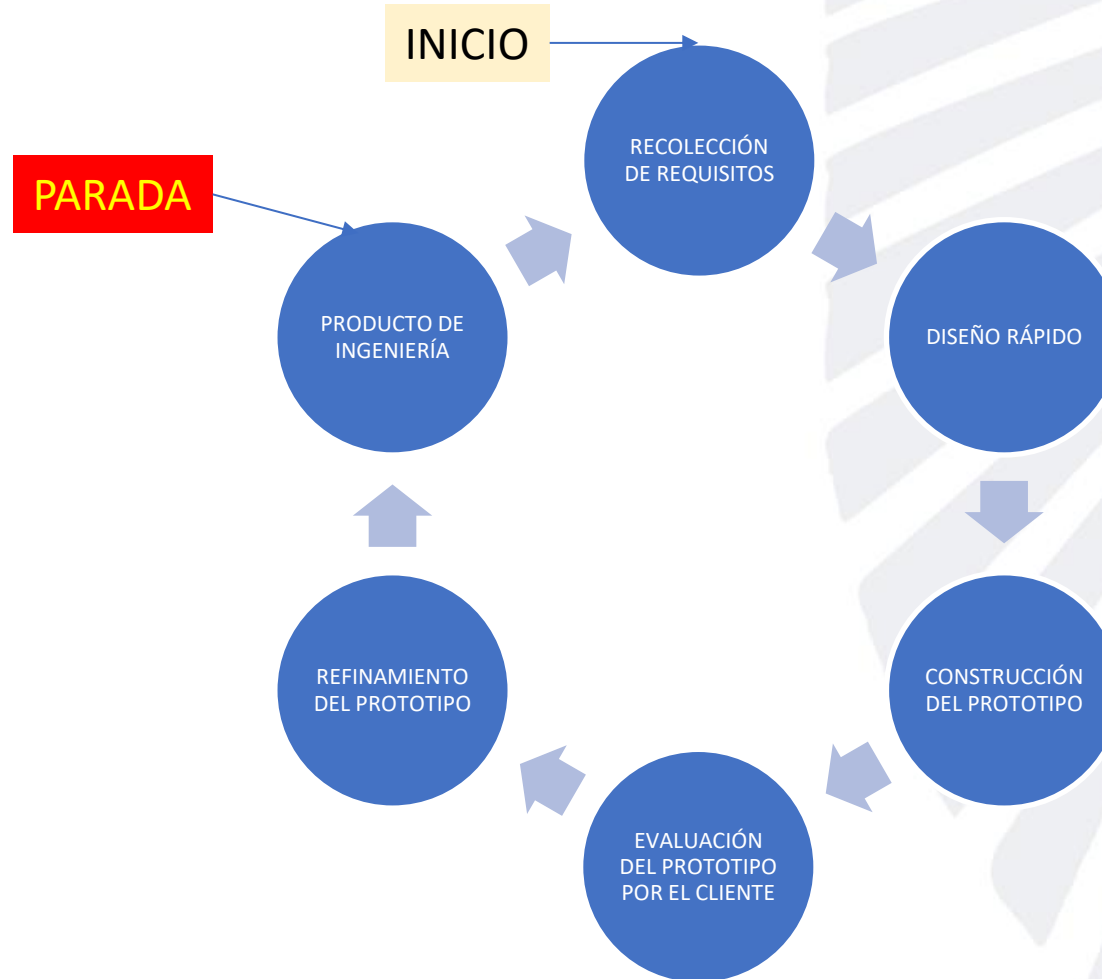
- Un modelo prototipo o modelo de desarrollo evolutivo es utilizado principalmente en el desarrollo de software para ofrecer al usuario una visión previa de cómo será el programa o sistema.
- Se le dice desarrollo evolutivo porque evoluciona hasta convertirse en el producto final.

CICLO DE VIDA EVOLUTIVO

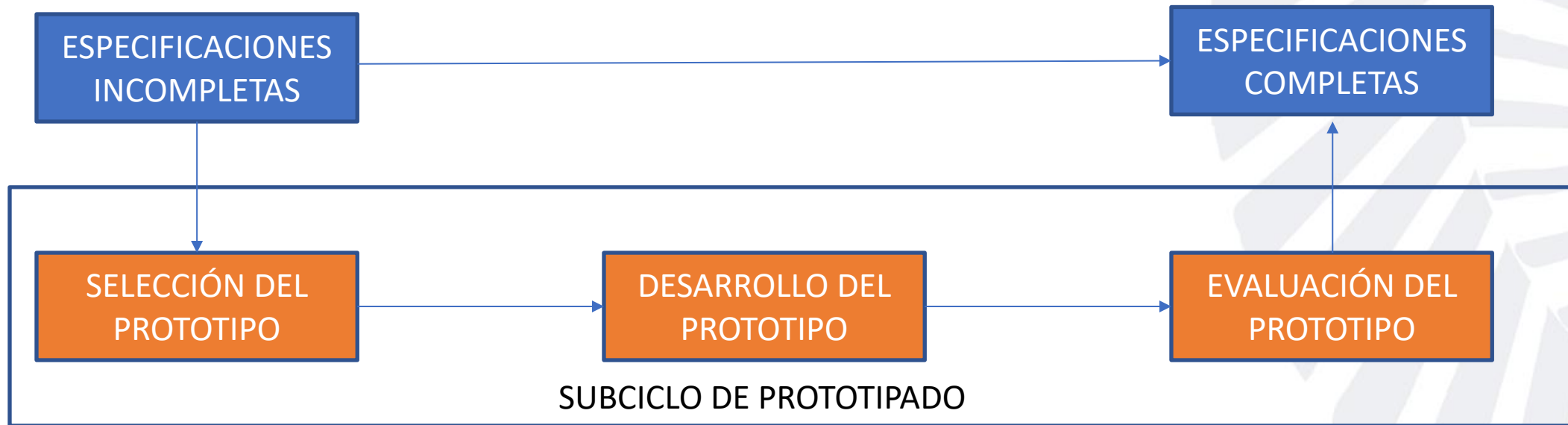
Resulta útil cuando:

- No se tienen todos los requerimientos al inicio
- Cuando el usuario no necesita todas las funcionalidades desde el inicio del proyecto

Metodologías para el Desarrollo de Software



CICLO DE VIDA POR PROTOTIPOS



- CARACTERÍSTICAS DEL MODELO PROTOTIPO:
 - No modifica el flujo del ciclo de vida
 - Reduce el riesgo de construir productos que no satisfagan las necesidades de los usuarios
 - Reduce costos y aumenta la probabilidad de éxito
 - Exige disponer de las herramientas adecuadas
 - No presenta calidad, ni robustez (es una cáscara)
 - Una vez identificados todos los requisitos mediante el prototipo, se construye el producto de ingeniería

CICLO DE VIDA POR PROTOTIPOS

Se utilizan para validar los requerimientos del usuario en cualquier ciclo de vida

Nos permiten suavizar la transición entre los requerimientos iniciales y finales que surgen en un proyecto con grandes dimensiones

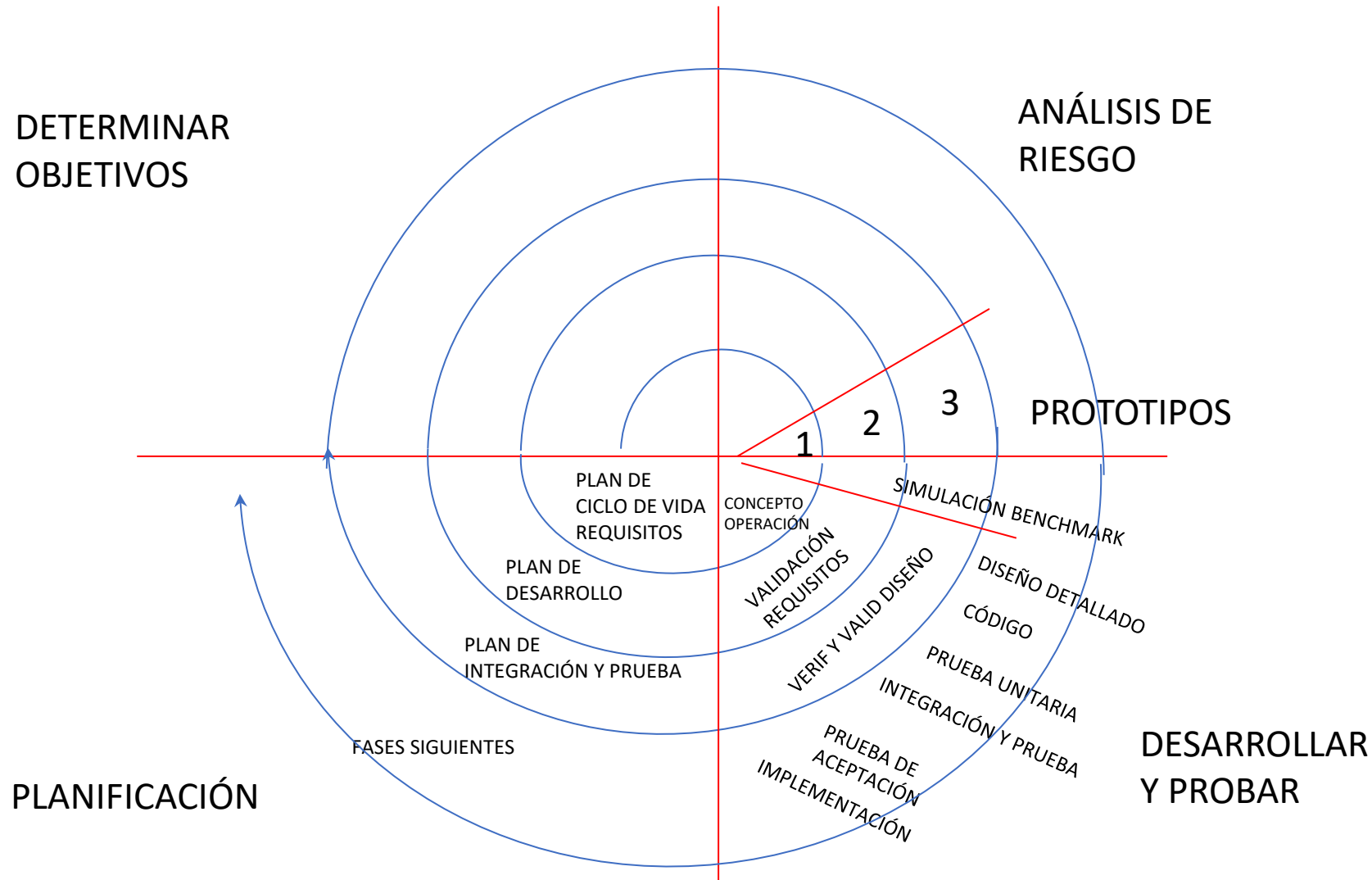
MODELO ESPIRAL

Qué es el método de desarrollo por espiral?

- Es un enfoque de desarrollo de software que puede ser considerado como una respuesta a los inconvenientes del desarrollo en cascada.
- Describe el ciclo de vida de un software por medio de espirales, que se repiten hasta que se puede entregar el producto terminado.

MODELO ESPIRAL

- **Fase 1:** definición de objetivos y alternativas y descripción de las condiciones generales.
- **Fase 2:** valoración de las alternativas.
- **Fase 3:** desarrollo y revisión del resultado intermedio.
- **Fase 4:** planificación del siguiente ciclo.



- **MODELO ESPIRAL**
 - Trata de mejorar los ciclos de vida clásicos y prototipos.
 - Permite acomodar otros modelos
 - Incorpora objetivos de calidad y gestión de riesgos
 - Elimina errores y alternativas no atractivas al comienzo
 - Permite iteraciones, vuelta atrás y finalizaciones rápidas
 - Cada ciclo empieza identificando:
 - Los objetivos de la porción correspondiente
 - Las alternativas
 - Restricciones
 - Se completa con una revisión que incluye todo el ciclo anterior y el plan para el siguiente

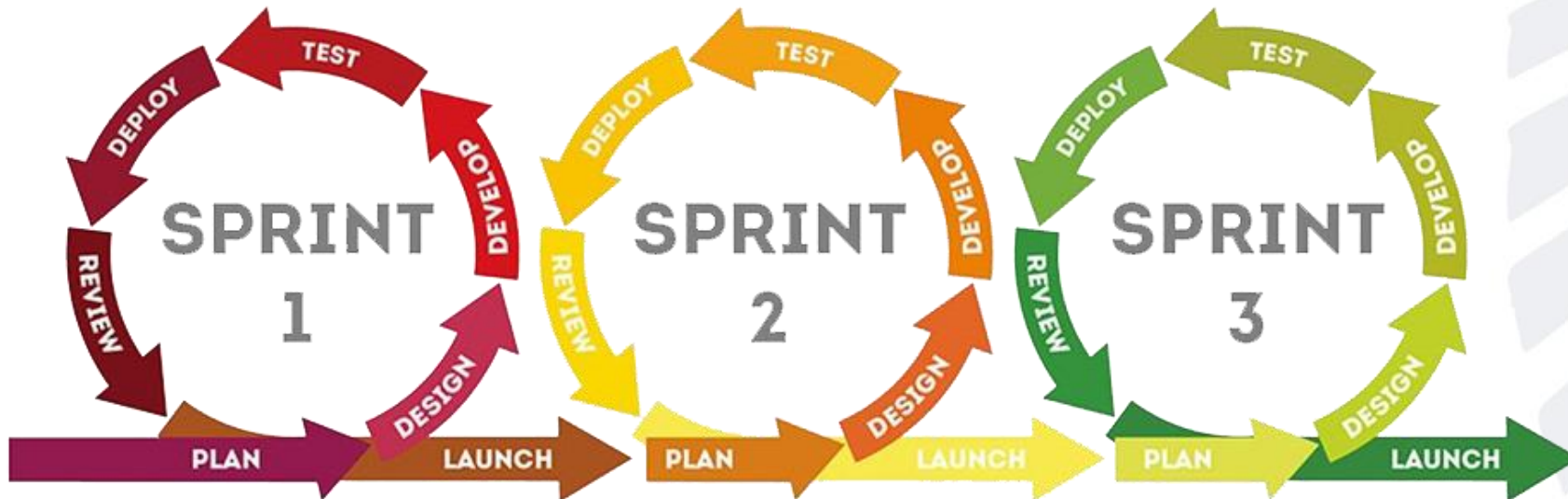
- DIFERENCIA ENTRE EL MODELO ESPIRAL Y LOS TRADICIONALES
 - Reconocimiento explícito de las diferentes alternativas.
 - Identificación de riesgos para cada alternativa desde el comienzo.
 - Al dividir el proyecto en ciclos, al final de cada uno existe un acuerdo para los cambios que hay que realizar en el sistema.
 - El modelo se adapta a cualquier tipo de actividad adicional

Metodologías
para el
Desarrollo de
Software

- **Agile** no es una metodología como el resto, sino que es un marco de trabajo (framework)
- Se basa en el manifiesto ágil.
<https://agilemanifesto.org/iso/es/manifesto.html>
- Permite adaptar la forma de trabajo al entorno de la organización, consiguiendo flexibilidad, rapidez en la respuesta.
- Mejora la satisfacción del cliente dado que se involucra y compromete en cada etapa del desarrollo del proyecto; se informará al cliente sobre los progresos, De ese modo, el cliente puede sumar su experiencia para optimizar las características del producto final. Se pueden evitar así numerosos malentendidos dado que el cliente poseerá en todo momento una completa visión del estado del producto.

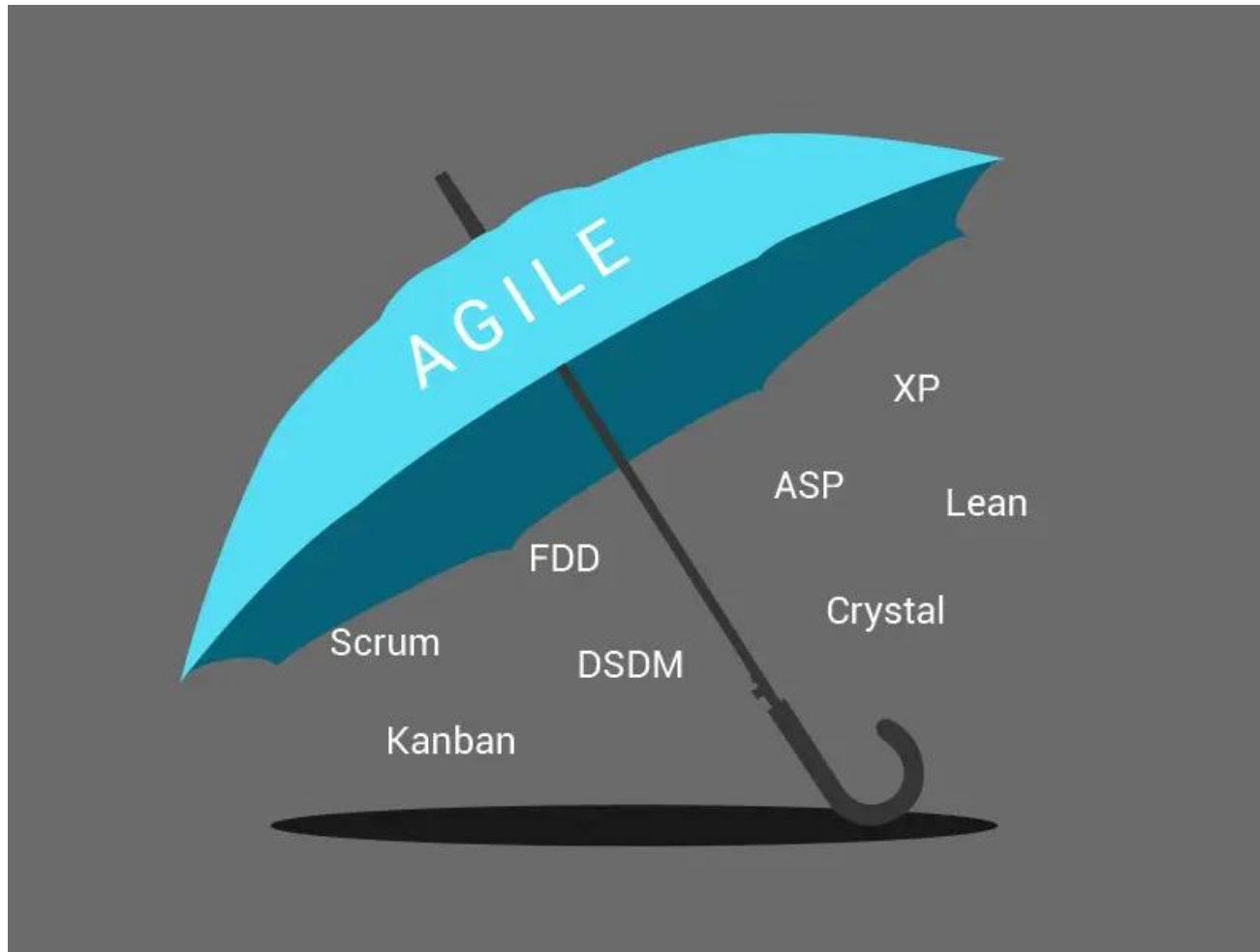
METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

AGILIDAD



- Es una forma de construir iterativa.
- Divide el desarrollo en Sprints

METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

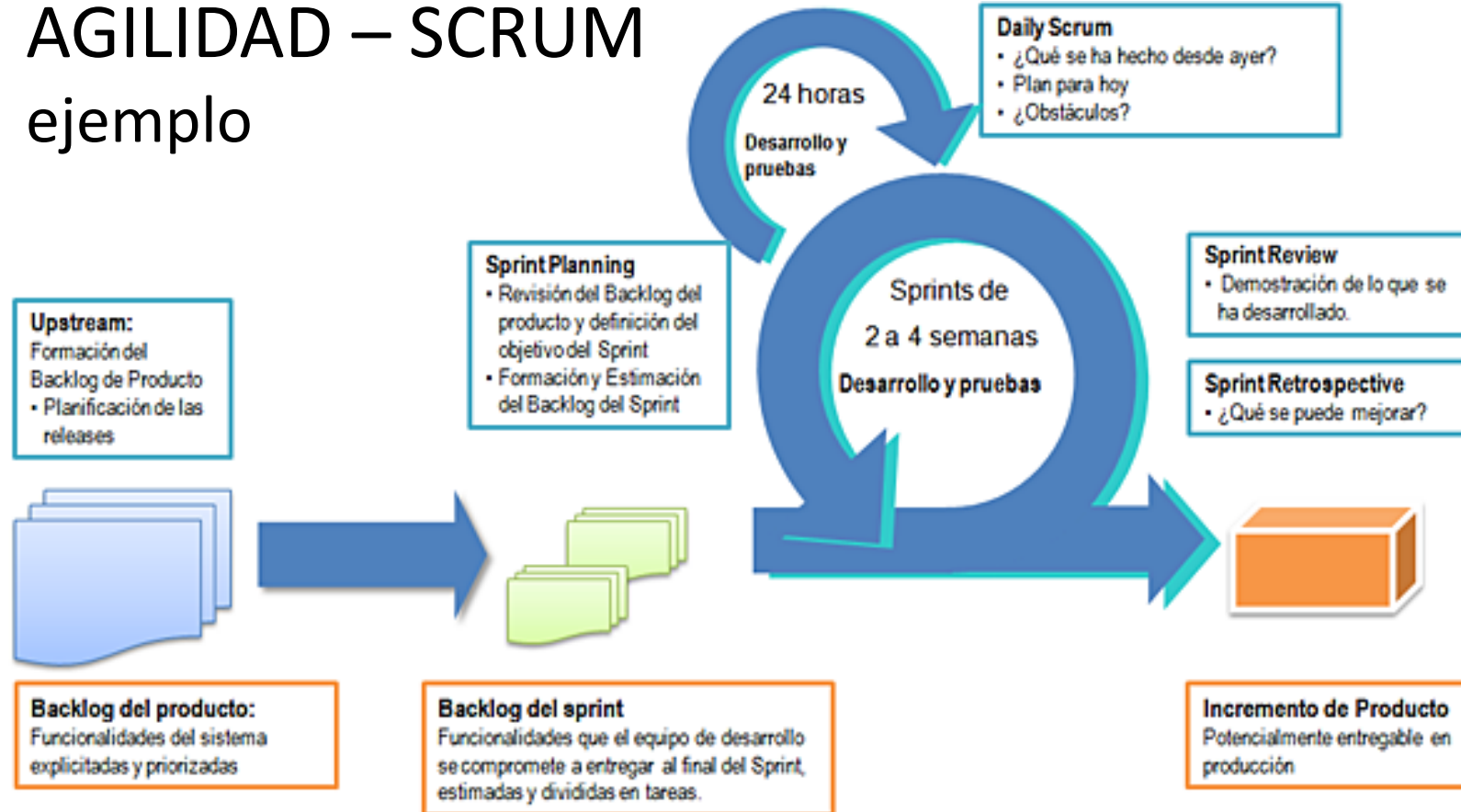


AGILIDAD

Existen distintos Marcos en los cuales basarse para el desarrollo ágil de proyectos.

METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

AGILIDAD – SCRUM ejemplo



El ciclo de vida es el siguiente:

1. El Product Owner redacta las User Stories y las sitúa en el Product Backlog.
2. El Product Owner prioriza estas User Stories y ordena el Product Backlog en consecuencia.
3. El equipo Scrum se junta en la reunión de planificación del Sprint, con el objetivo de establecer la lista de las User Stories que se tratarán durante el Sprint. Esto forma el Sprint Backlog y a continuación se descomponen en tareas por el equipo de desarrollo.
4. El Sprint puede comenzar con una iteración de 2, 3 o 4 semanas.
5. El equipo se reúne diariamente para realizar la daily (reunión diaria).
6. Como consecuencia del Sprint, obtenemos un producto potencialmente entregable que forma parte de una demo durante la revisión del Sprint.
7. El ciclo termina con la retrospectiva del Sprint.

METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

Metodologías Tradicionales



Metodologías Ágiles



METODOLOGÍAS PARA EL DESARROLLO DE SOFTWARE

	Methodology		
Characteristic	SDLC	RAD	Agile
Overall Flow	Structured	Structured	Incremental
Analysis	Methodical	Methodical	Iterative
Design	Methodical	Interactive	Iterative
Development	Sequential	Interactive	Iterative
Testing	Follows Development	Combined with Development	Combined with Development



UNIVERSIDAD NACIONAL
RAÚL SCALABRINI ORTIZ
SAN ISIDRO

@unsanisisidro



www.unso.edu.ar

Diagrama de Flujo

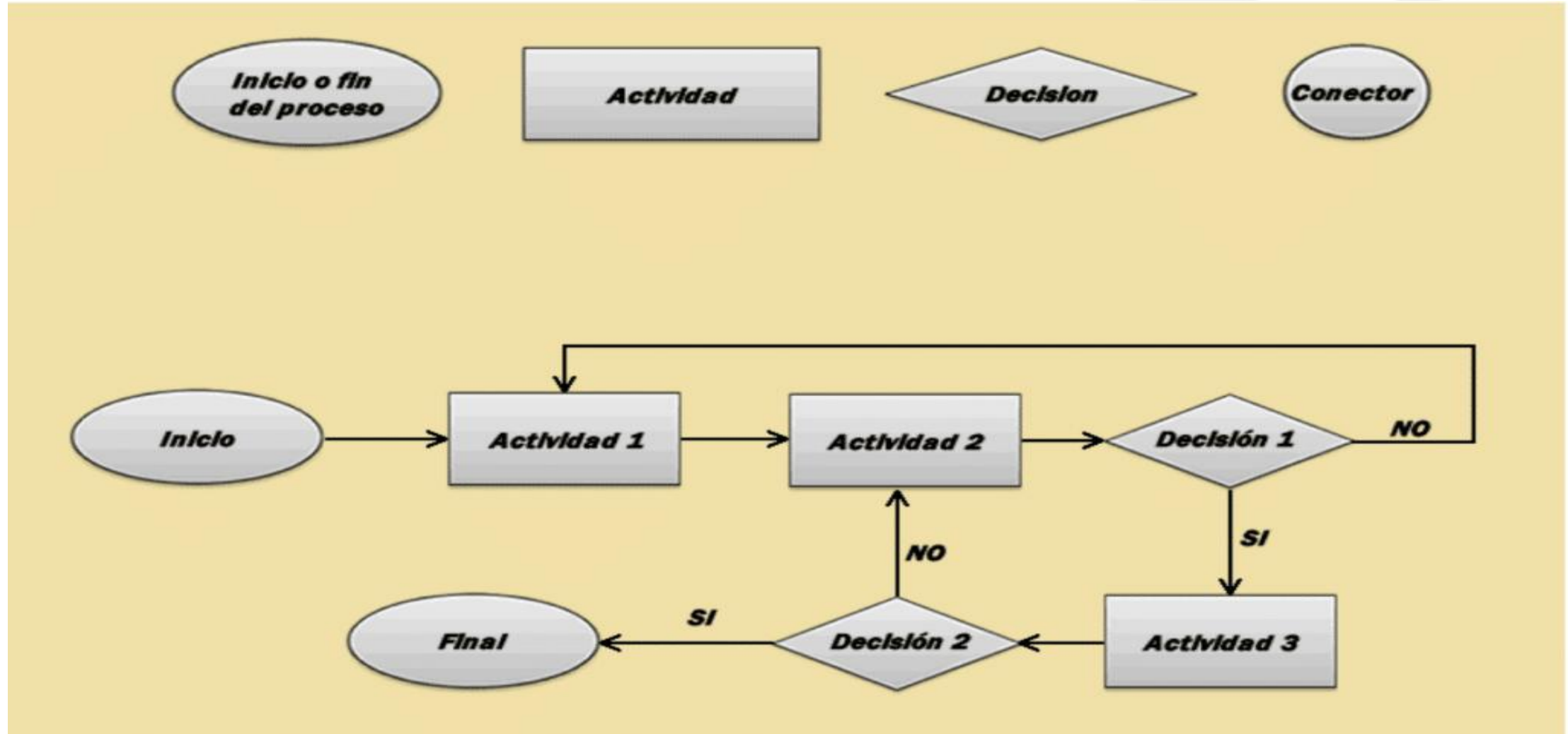
El **diagrama de flujo** o **flujograma** es la representación gráfica de un algoritmo o proceso.

- Se utilizan símbolos que representan los pasos y decisiones del algoritmo, y representan el flujo de ejecución mediante flechas.
- Determina el comportamiento requerido de un proceso independientemente de la tecnología.
- UML, BPMN, ISO 5807

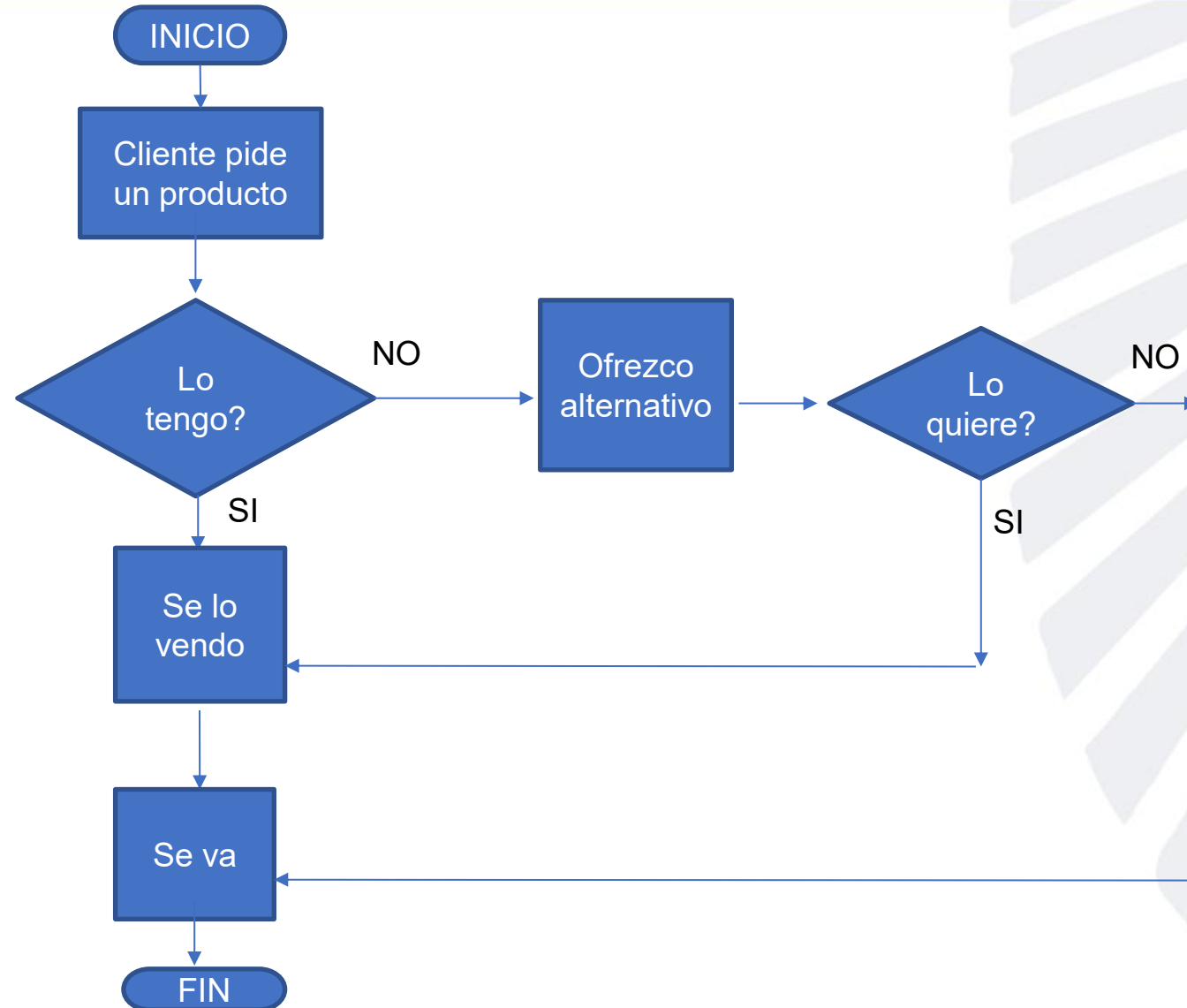
MODELADO DIAGRAMA DE FLUJO

Símbolo	Utilidad o significado
	Línea de flujo o flecha, usado para mostrar la continuidad y orden del proceso.
	Usado para indicar el inicio o fin del proceso o subproceso.
	Indica una actividad del procesos en el diagrama.
	Usado para indicar un decisión de dos opciones (si, no o verdadero, falso).
	Entrada y salida de datos al proceso.
	Documentos del proceso.
	Usado pasa representar un enlace dentro de otra parte del proceso, este lleva una letra o numero en el centro que es con que empieza el enlace.
	Usado para representar demora en la secuencia de una actividad.
	Usado para representar dos partes del diagrama en diferentes paginas.

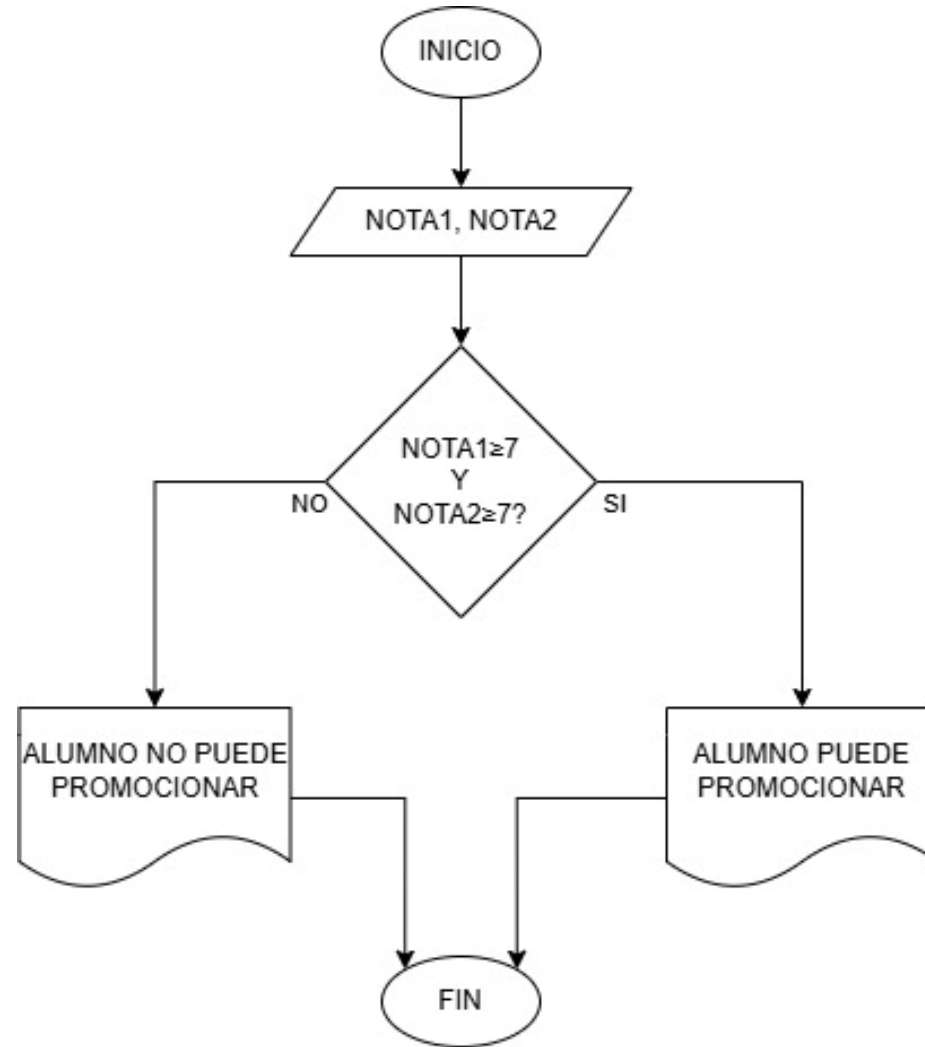
MODELADO DIAGRAMA DE FLUJO



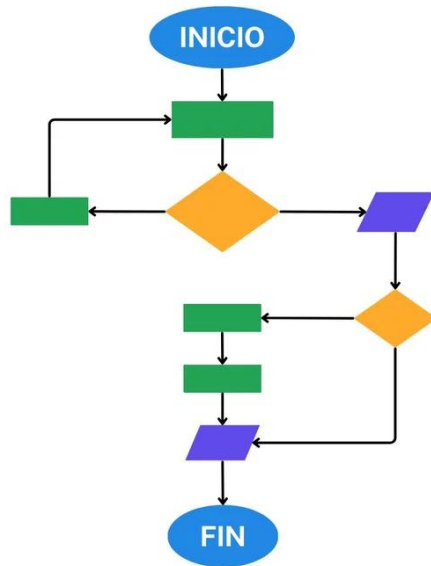
EJEMPLO PROCESO DIAGRAMA DE FLUJO



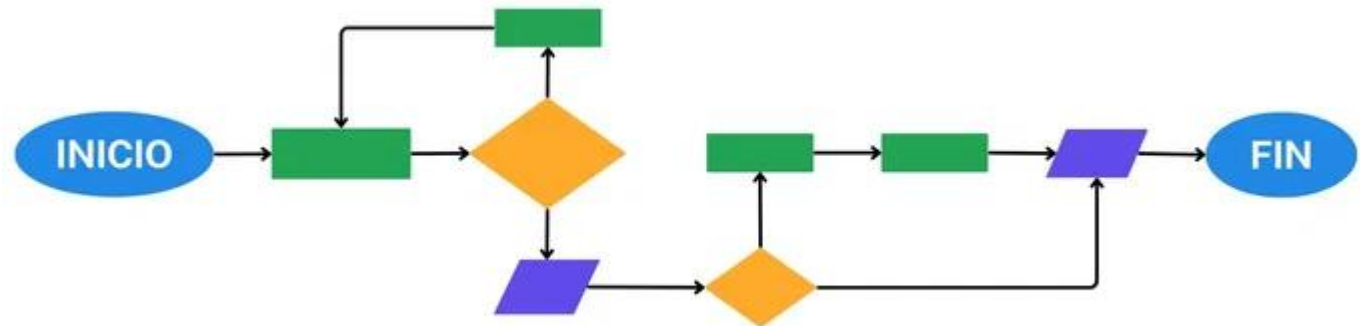
EJEMPLO ALGORITMO DIAGRAMA DE FLUJO



VERTICAL



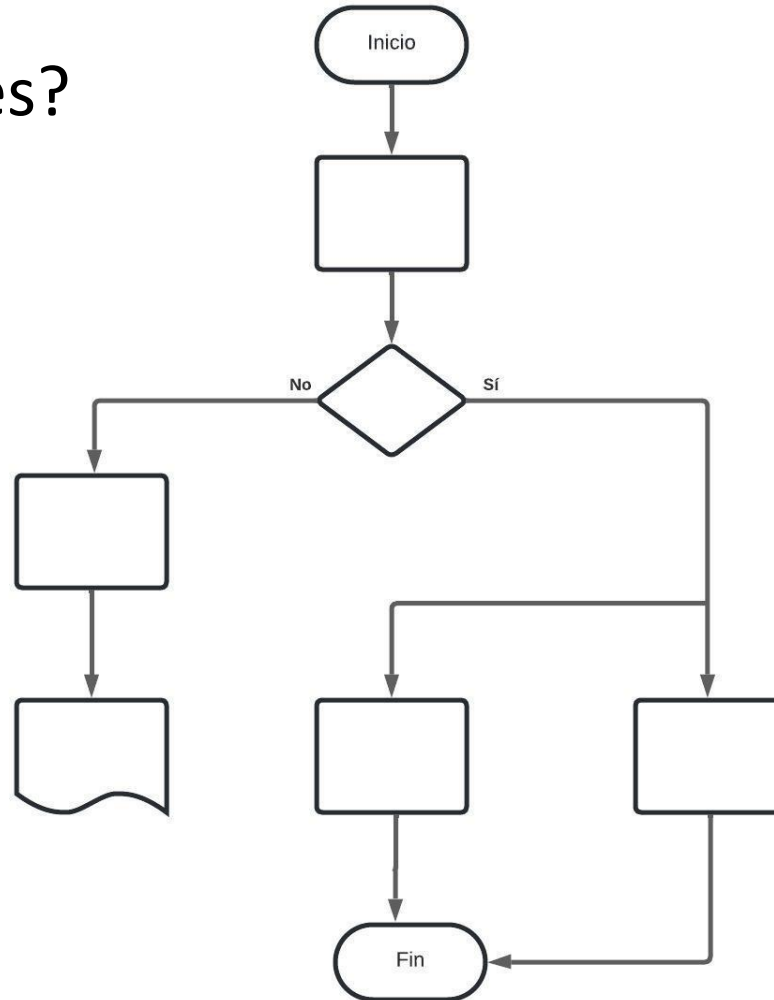
HORIZONTAL



Reglas Para la Construcción

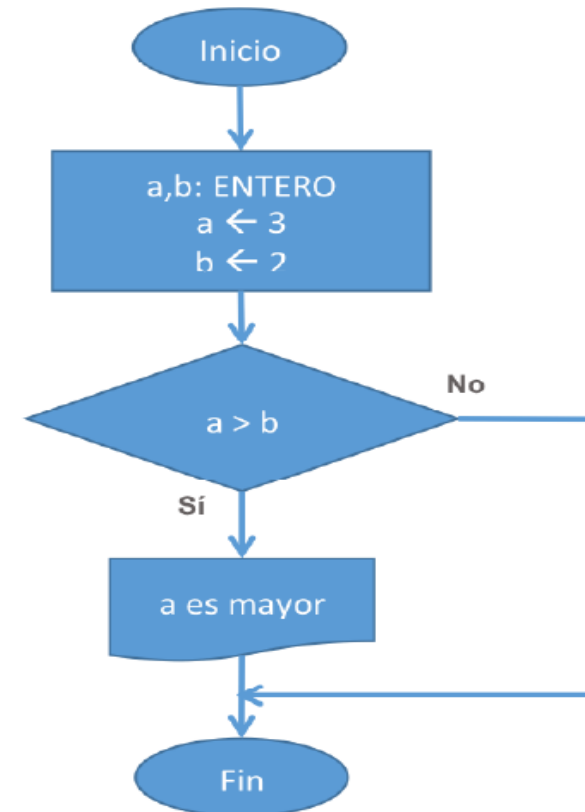
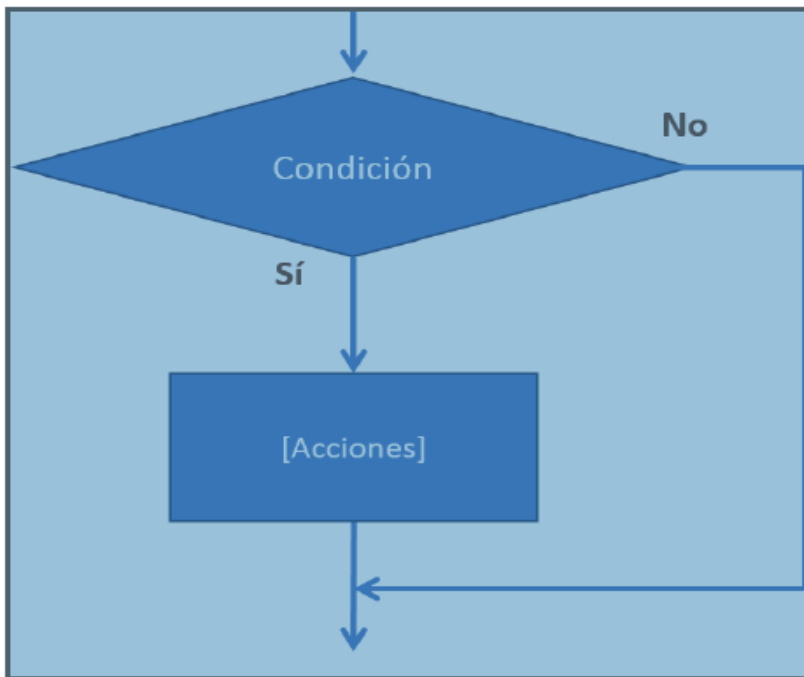
1. Todos los símbolos tienen que estar conectados
2. A un símbolo de proceso pueden llegarle varias líneas
3. A un símbolo de decisión pueden llegarle varias líneas, pero sólo saldrán dos (Sí o No, Verdadero o Falso).
4. A un símbolo de inicio nunca le llegan líneas.
5. De un símbolo de fin no parte ninguna línea.

¿Qué errores le ves?



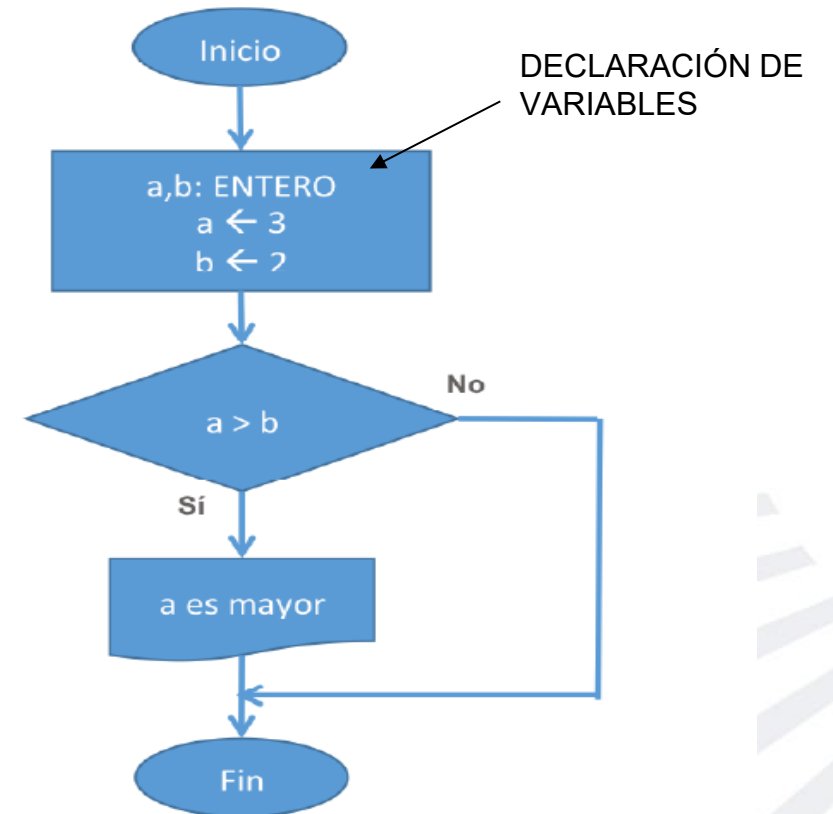
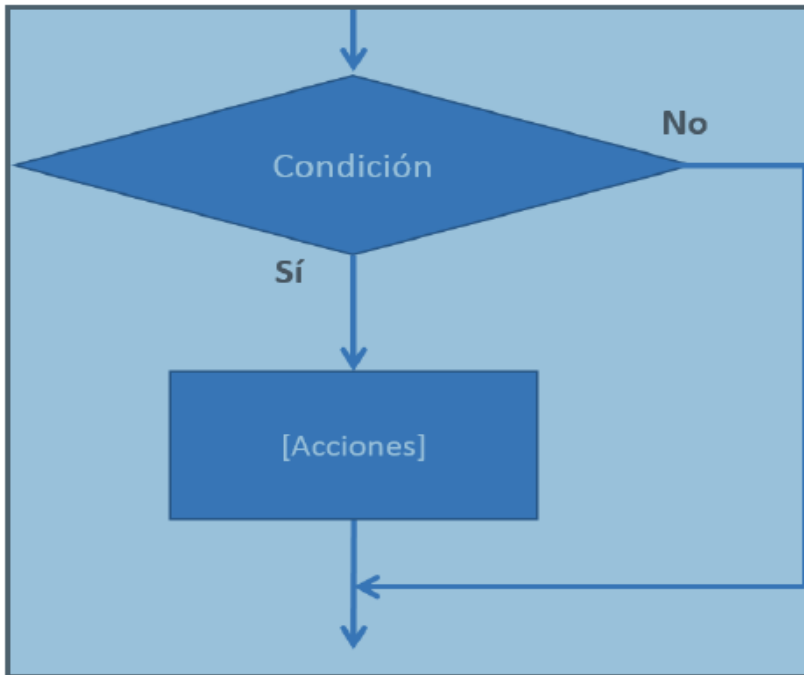
ESTRUCTURA DE CONTROL DE FLUJO

- CONDICIONAL



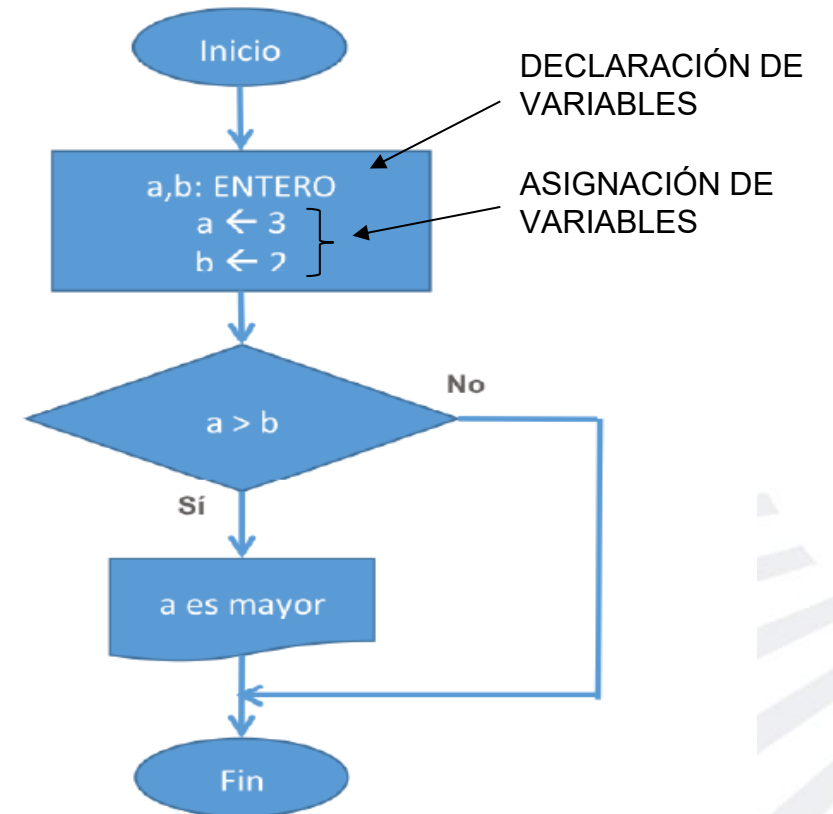
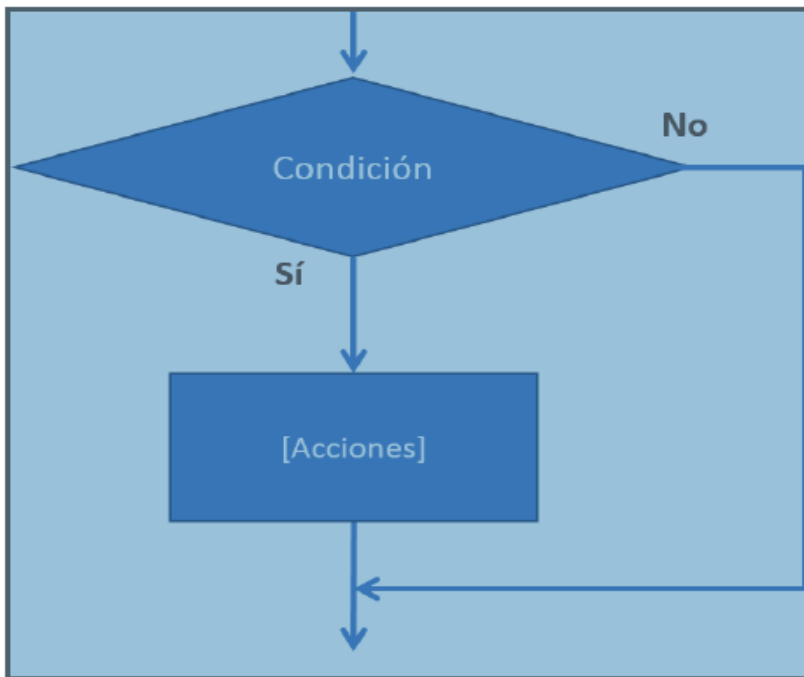
ESTRUCTURA DE CONTROL DE FLUJO

- CONDICIONAL



ESTRUCTURA DE CONTROL DE FLUJO

- CONDICIONAL



EJEMPLO CONDICIONAL DIAGRAMA DE FLUJO

Pseudocódigo

1. Inicio

2. Inicializar variables: $A = 0$, $B = 0$

3. Solicitar la introducción de dos valores distintos

4. **Leer** los dos valores

5. Asignarlos a las variables A y B

6. **Si** $A = B$ **Entonces** vuelve a 3 porque los valores deben ser distintos

7. **Si** $A > B$ **Entonces**

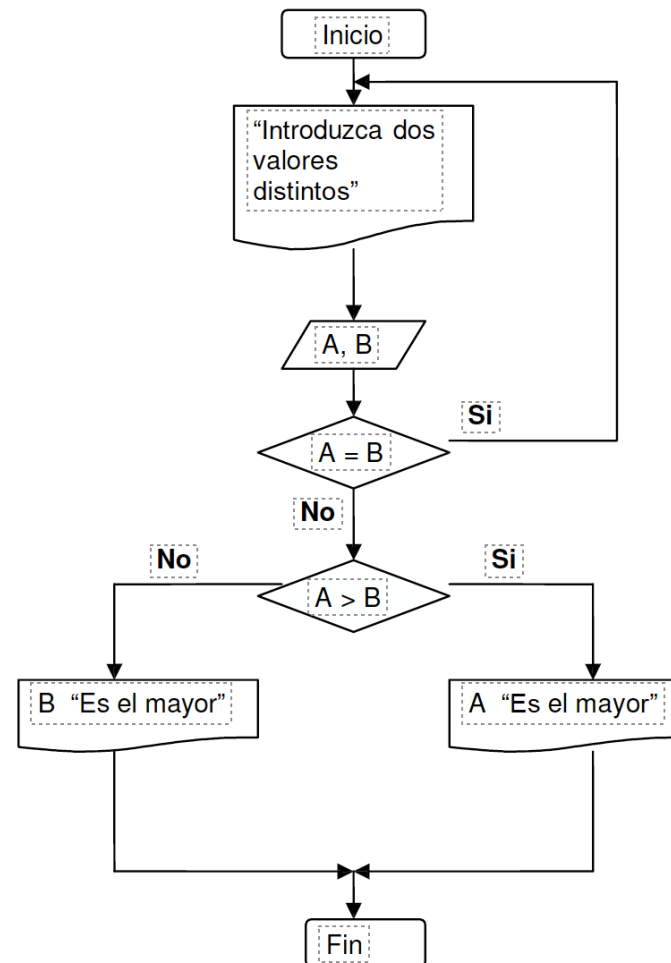
Escribir A, "Es el mayor"

8. **De lo contrario:** **Escribir** B, "Es el mayor"

9. **Fin_Si**

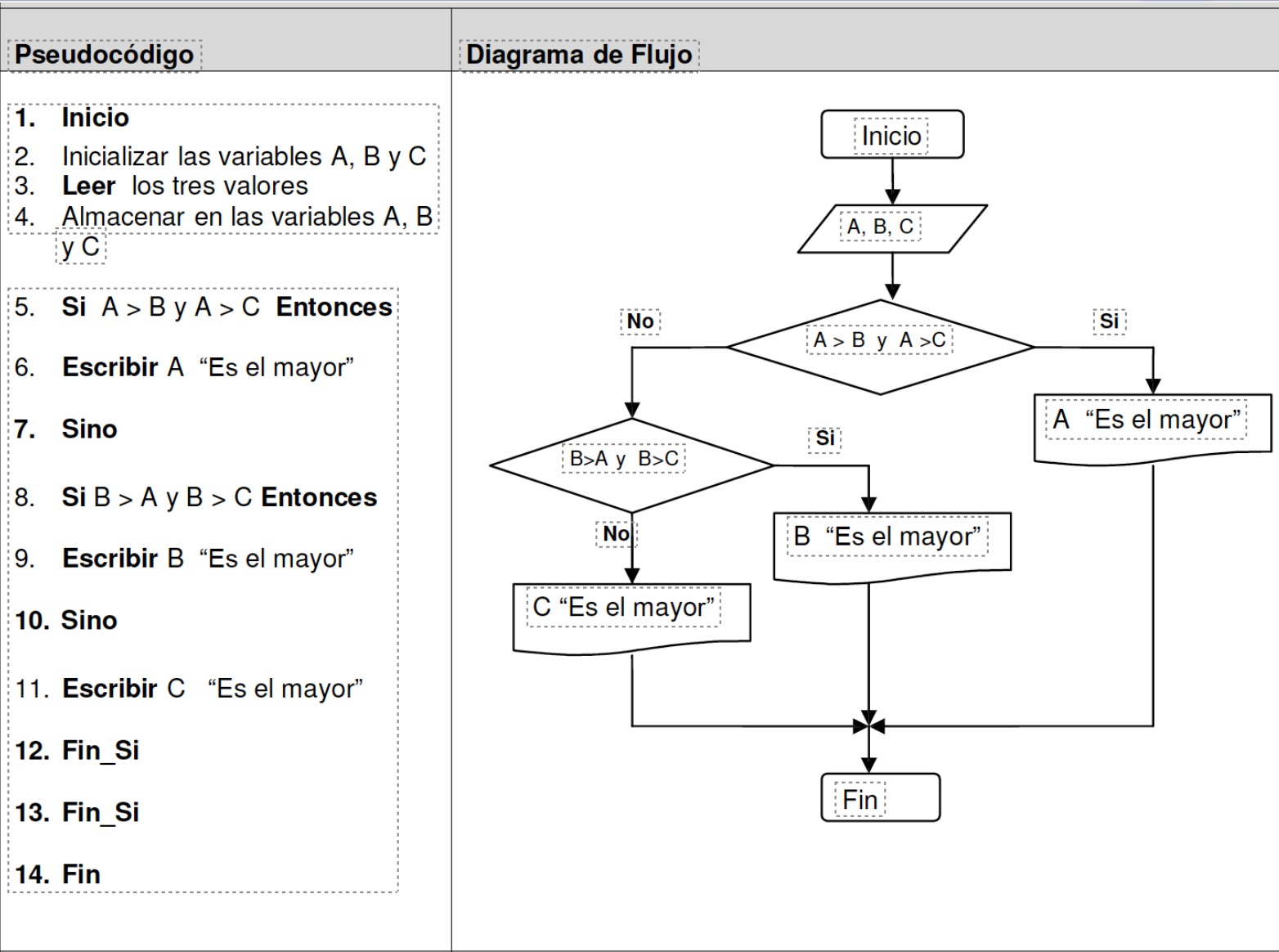
10. **Fin**

Diagrama de Flujo



EJEMPLO CONDICIONAL

DIAGRAMA DE FLUJO

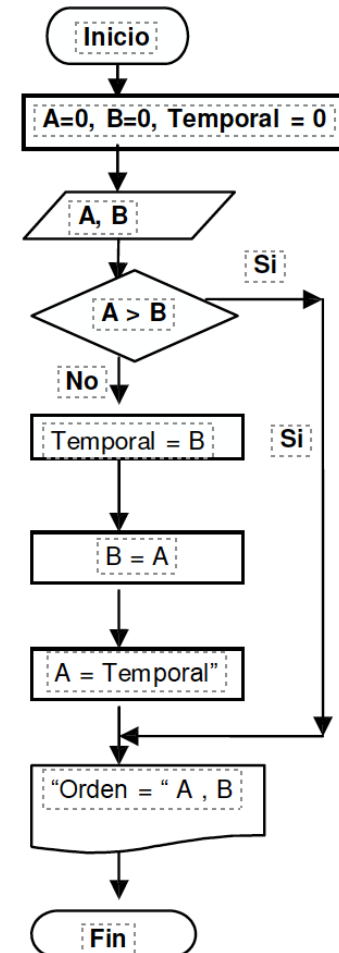


EJEMPLO CONDICIONAL DIAGRAMA DE FLUJO

Pseudocódigo

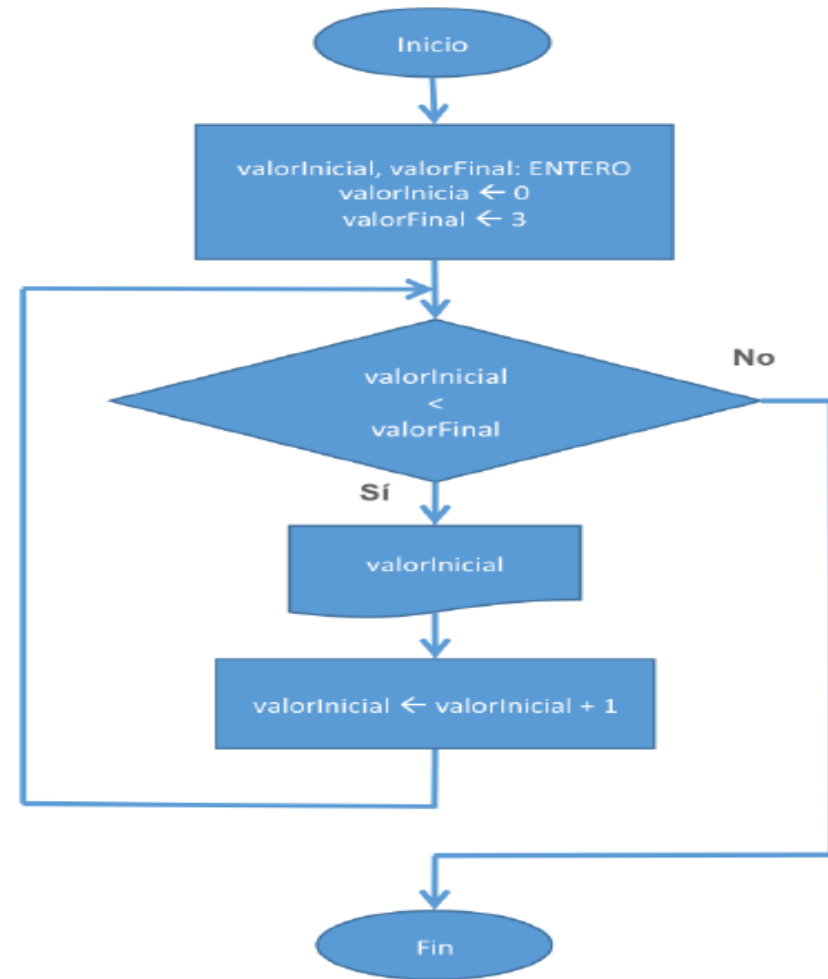
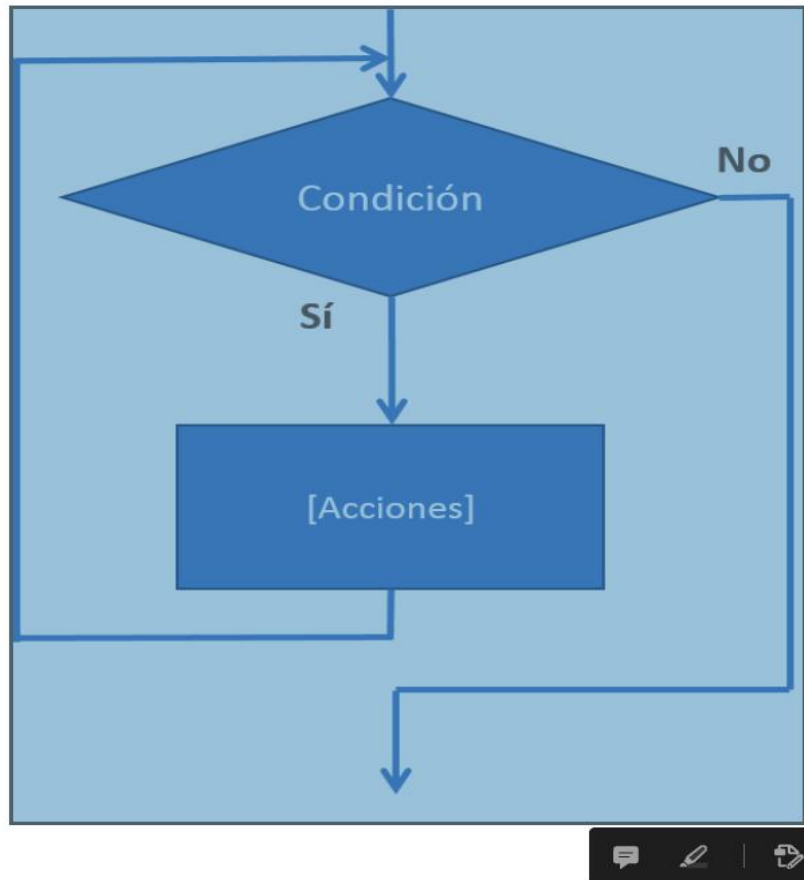
1. **Inicio**
2. Declaración de Variables:
A = 0, B = 0, Temporal = 0
3. **Leer** A y B
4. **Si** A < B **Entonces**
5. Asignar a Temporal = B
6. Asignar a B = A
7. Asignar a A = Temporal
8. **Si no** (De lo contrario)
9. **Fin_Si**
10. Escribir "Orden = ", A, B
11. **Fin**

Diagrama de Flujo



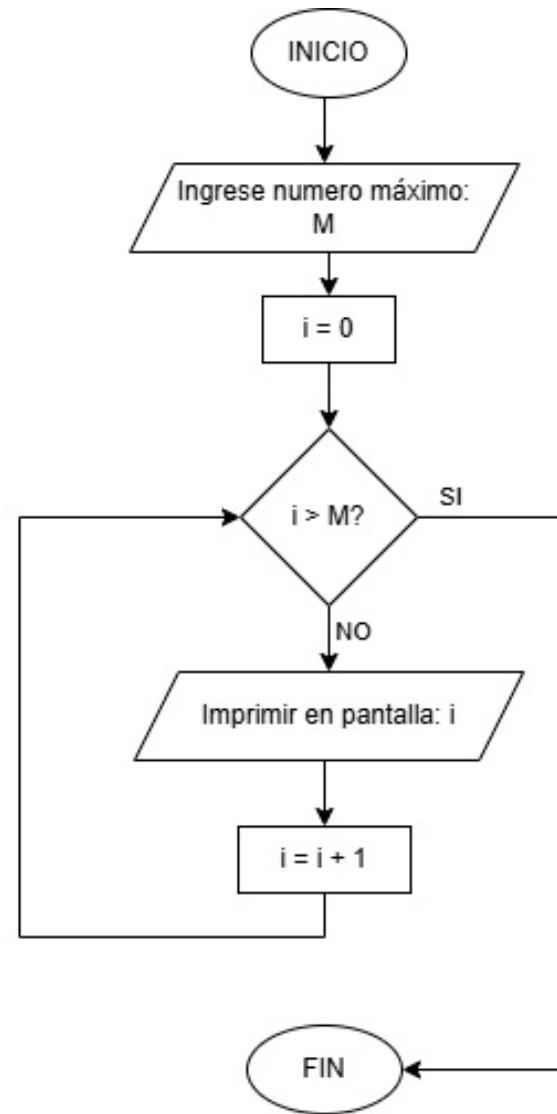
ESTRUCTURA DE CONTROL DE FLUJO

- ITERATIVA

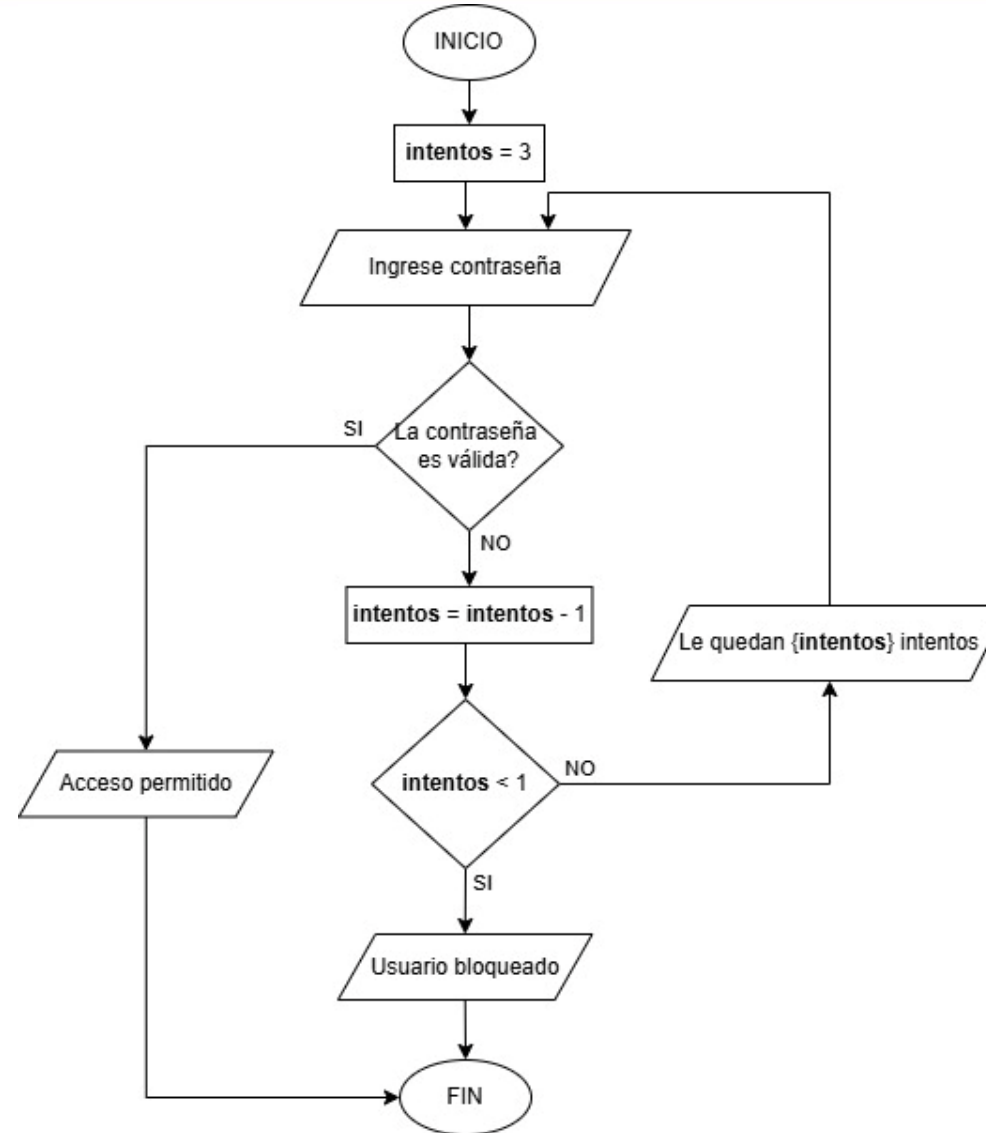


```
//>>> 0  
//>>> 1
```

EJEMPLO ITERATIVA DIAGRAMA DE FLUJO



EJEMPLO ITERATIVA DIAGRAMA DE FLUJO



PREGUNTAS

